Practical Excel & Financial Modeling

Practical, Hands-On, Job-Ready — India-first + Global

Real formulas, queries, entries & tools — closing the gap between what an MBA teaches and what finance employers pay for

Contents

Excel: the #1 finance skill and the gap it exposes

What it is & where it's used

Excel is the universal spreadsheet — a grid of cells that becomes a calculator, a database, a reporting tool, and a modeling engine depending on how you use it. In finance it is not "one tool among many"; it is the default surface where numbers get worked. SAP, Oracle, Tally, Zoho Books, and every bank portal ultimately export to Excel because that's where humans reason about the data.

Every finance role touches it, but differently:

Role	What they do in Excel daily
Accounts / AP-AR	Reconciliations, ageing schedules, ledger dumps, GST 2A/2B matching
Audit / assurance	Sampling, variance testing, tie-outs, TB-to-FS mapping
FP&A / analyst	Budgets, variance decks, rolling forecasts, KPI trackers
Investment banking / PE	3-statement models, DCF, LBO, comps
Taxation	Computation sheets, TDS workings, GST returns prep, 26AS reconciliation
Treasury	Cash flow forecasting, interest schedules, bank position sheets

If you can only bring one skill to a finance desk on day one, it is Excel. Everything else — Tally, SQL, Power BI — sits around it.

The gap: why companies want this (and college didn't teach it)

An MBA teaches you *what* NPV, WACC, and working capital mean. A CA course teaches you *what* the standards require. Neither teaches you to **build the sheet that a manager trusts at 11 pm before a board meeting.** That build skill is the gap.

Concretely, college leaves you unable to:

- Pull the right value from a 40,000-row ledger dump **without scrolling** (lookups, not eyeballing).
- Build a formula that **doesn't break** when a row is inserted (absolute vs relative references, structured tables).
- Make one input change and have the **whole model recalculate correctly** (linked, not hardcoded).
- Produce output a reviewer can **audit in 30 seconds** (colour conventions, no buried constants).
- Do all of the above **fast, keyboard-only, under time pressure.**

Employers pay for the person who receives a messy export and returns a clean, correct, self-checking answer unaided. That's the difference between "I've used Excel" and "job-ready Excel."

What "proficient" looks like

The concrete bar employers test for — a proficient person can, unaided:

1. **Navigate and clean** a raw export: remove duplicates, `TRIM` / `CLEAN` text, split columns, fix date-as-text.
2. **Look up and match** across sheets with `XLOOKUP` / `INDEX–MATCH`, and multi-criteria with `SUMIFS`.

3. **Build logic** with nested `IF`, `IFS`, `AND` / `OR`, and error-trap with `IFERROR`.
4. **Aggregate** thousands of rows via PivotTables in under two minutes.
5. **Model** with a clean inputs → calc → output structure and no hardcoded numbers inside formulas.
6. **Work keyboard-first** — `Ctrl+Arrow`, `Ctrl+Shift+Arrow`, `Alt+=`, `F4`, `Ctrl+;`.
7. **Self-check** — build a control total that must equal zero.

A rough ladder: Basic (formulas, formatting), Intermediate (lookups, pivots, charts — this is the *hiring minimum* for most accounts/analyst roles), Advanced (dynamic arrays, `LET` / `LAMBDA`, full 3-statement models).

Hands-on: how to actually do it

Lookups — use XLOOKUP, fall back to INDEX-MATCH.

```
=XLOOKUP(A2, Vendors[GSTIN], Vendors[VendorName], "Not found")

=INDEX(Vendors[VendorName], MATCH(A2, Vendors[GSTIN], 0))
```

`XLOOKUP` looks up `A2` in the GSTIN column and returns the vendor name; the 4th argument replaces the ugly `#N/A`. `INDEX-MATCH` does the same and works in every Excel version.

Multi-criteria sum — SUMIFS is the workhorse of accounts.

```
=SUMIFS(Ledger[Amount], Ledger[Party], "Acme Ltd", Ledger[Type], "Sales", Ledger[Date], "≥"&D
```

Sum of all Acme sales from 1-Apr-2025 onward. `COUNTIFS` and `AVERAGEIFS` follow the same pattern.

Conditional logic with error trapping.

```
=IFERROR(IF(D2>1000000, D2*0.1, D2*0.05), 0)
```

GST split from a tax-inclusive amount (intra-state, 18%).

```
=ROUND(A2/1.18*0.09, 2)   // CGST
=ROUND(A2/1.18*0.09, 2)   // SGST
```

Data cleaning that saves reconciliations.

```
=TRIM(CLEAN(A2))                // strip stray spaces + non-printing chars
=TEXT(A2,"0000000000")          // pad GSTIN/PAN keys before matching
=DATEVALUE(A2)                  // convert text-dates so they compare correctly
```

Dynamic arrays (Excel 365) — filter without a pivot.

```
=FILTER(Ledger, Ledger[Balance]>0)
=SORT(UNIQUE(Ledger[Party]))
```

The two keyboard reflexes that separate pros from beginners: `F4` to toggle `$` anchors (`A1` → `$A$1`), and `Alt + =` to auto-sum a column instantly.

Worked example / mini-project: GST 2B input-credit reconciliation

You have two sheets: **Books** (purchases you recorded) and **GSTR-2B** (what the portal says vendors filed). You must find invoices where credit is at risk.

Books (columns A–D): Invoice No, GSTIN, Taxable Value, GST in books. **2B** (columns A–D): Invoice No, GSTIN, Taxable Value, GST as per 2B.

Step 1 — build a match key in both sheets (E2, filled down):

```
=TRIM(B2)&"|"&TRIM(A2)
```

Step 2 — in **Books**, pull the 2B tax against each invoice:

```
=XLOOKUP(E2, '2B'!$E$2:$E$5000, '2B'!$D$2:$D$5000, "MISSING IN 2B")
```

Step 3 — flag the difference (G2):

```
=IF(F2="MISSING IN 2B","BLOCK CREDIT", IF(ROUND(D2-F2,0)=0,"OK","MISMATCH ₹"&TEXT(D2-F2,"0")))
```

Sample result:

Invoice	GSTIN	GST books	GST 2B	Flag
INV-101	29AABCU...	18,000	18,000	OK
INV-102	27AAAC...	9,000	0	MISSING IN 2B → BLOCK CREDIT
INV-103	24AACC...	12,600	11,700	MISMATCH ₹900

Step 4 — the control total (single cell, must show a real number, not error):

```
=SUMIF(G:G,"BLOCK CREDIT",D:D)   // total ITC you cannot claim this month
```

That last cell is what a manager actually reads. You've turned two raw dumps into a decision: how much input credit to reverse. This is exactly the kind of task a fresher gets in week one at a tax or audit firm.

How it's tested

In the interview (verbal): - "Difference between `VLOOKUP` and `INDEX-MATCH` ? Why might `XLOOKUP` be better?" - "What does `F4` do while editing a formula?" (Expect: toggles absolute/relative reference.) - "How

would you find duplicate invoice numbers?" (Answer: `COUNTIF(range,cell)>1`, or Remove Duplicates, or a pivot.) - "A `SUMIFS` returns 0 but you expect a number — how do you debug?" (Text vs number, trailing spaces, date-as-text.)

The practical test (this is where offers are decided): a **timed 30–60 minute Excel test.** Typical brief:

> "Here's a 5,000-row sales export. Build a summary by region and month, flag any order above ₹5 lakh, look up the salesperson from the second tab, and give me total revenue with a control check. You have 40 minutes."

They watch (or infer from your file): Did you use lookups or copy-paste? Did you hardcode? Does it break if a row is added? Is there a self-check? For accounting roles the case is often **"reconcile these two ledgers"** or **"close these books and produce a TB."** Speed and correctness both count — and using the keyboard, not the mouse, visibly signals fluency.

Common mistakes & how pros avoid them

Mistake	Why it hurts	Pro habit
Hardcoding numbers inside formulas (`=B2*0.18`)	Nobody can find or change the assumption	Put 18% in a labelled input cell; reference it
Manual `VLOOKUP` with a counted column index	Breaks the instant a column is inserted	`XLOOKUP` / `INDEX-MATCH` on named columns
Forgetting `$` anchors when copying	Formula drifts, silent wrong answers	Hit `F4`; know what to lock
No error trapping	`#N/A` cascades and breaks totals	Wrap in `IFERROR`
No control total	Errors ship undetected	One cell that must equal zero
Mixing inputs, calcs, outputs on one sheet	Un-auditable	Blue = input, black = formula; separate zones
Merged cells everywhere	Kills sorting, pivots, references	"Center Across Selection" instead
Volatile mega-formulas nobody can read	Un-reviewable, un-maintainable	Break into steps or use `LET` to name parts

The colour convention (blue font for hardcoded inputs, black for formulas) is a genuine industry standard — reviewers scan for blue to find every assumption in seconds.

Learn-it roadmap & resources

Realistic time-to-proficiency (from an MBA/CA base, practising ~1 hr/day):

Level	Time	You can then…
Basic → Intermediate	3–4 weeks	Lookups, `SUMIFS`, pivots, clean charts — clears most job screens
Intermediate → job-ready	6–8 weeks	Handle real dumps, reconcile unaided, pass timed tests
Advanced	3–4 months	Dynamic arrays, `LET` / `LAMBDA`, full 3-statement models

Resources: - *Free:* Microsoft's own Excel training (support.microsoft.com), ExcelJet's formula reference, Chandoo.org, Kenji Explains and Leila Gharani on YouTube. For India-specific GST/TDS sheets, the CBIC and income-tax portals give real return formats to rebuild. - *Paid / certification:* Microsoft Office Specialist (MOS): Excel Associate/Expert — the globally-recognised cert; Corporate Finance Institute (CFI) Excel courses; Wall Street Prep / BIWS for modeling. In India, an MOS Excel Expert badge on a resume meaningfully de-risks you to a hiring manager. - *Practice, not watching:* download a bank statement and a ledger, and force yourself to reconcile them keyboard-only. Repetition on real data beats any course.

Turn off the mouse for an hour a day. That single constraint builds the fluency employers actually test.

Quick-reference

Task	Formula / shortcut
Lookup (modern)	`=XLOOKUP(val, lookup_col, return_col, "NA")`
Lookup (any version)	`=INDEX(ret, MATCH(val, look, 0))`
Multi-criteria sum	`=SUMIFS(sum, c1, k1, c2, k2)`
Count with condition	`=COUNTIFS(range, criteria)`
Duplicate check	`=COUNTIF(A:A, A2)>1`
Error trap	`=IFERROR(formula, 0)`
Clean text key	`=TRIM(CLEAN(A2))`
Text-date → date	`=DATEVALUE(A2)`
GST 18% split	`=ROUND(amt/1.18*0.09,2)` each for CGST & SGST
Filter (365)	`=FILTER(tbl, tbl[col]>0)`
Unique list	`=SORT(UNIQUE(range))`
Auto-sum	`Alt + =`
Toggle absolute ref	`F4`
Today's date (static)	`Ctrl + ;`
Jump to data edge	`Ctrl + Arrow`
Select to edge	`Ctrl + Shift + Arrow`
Remove duplicates	Data tab → Remove Duplicates
PivotTable	`Alt, N, V` → drag fields

GST rate reminder (intra-state): total rate splits equally into CGST + SGST (e.g. 18% = 9% + 9%); inter-state is a single IGST at the full rate. Colour code: **blue = input, black = formula.** Always end with one **control cell that must equal zero.**

Excel Foundations Pros Use

What it is & where it's used

Excel is the default operating system of finance. Not because it's the best tool for every job, but because it is the one tool every analyst, controller, banker, auditor, and CFO can open on any machine and immediately read. In an Indian finance job you will live inside Excel for MIS reports, budget vs actuals, receivable ageing, GST reconciliations (GSTR-2B vs purchase register), invoice trackers, DCF and valuation models, and the monthly deck that goes to management.

"Foundations pros use" means the boring, invisible layer that separates a fast analyst from a slow one: working without touching the mouse, structuring a workbook so it can be audited, formatting numbers so a ₹ figure reads correctly at a glance, and knowing exactly when a reference should lock. Roles that test this on day one: **FP&A / MIS analyst, audit associate (Big 4 and mid-tier), investment banking / equity research associate, accounts executive, treasury, and tax analyst.** Every one of them assumes you already have this. Nobody trains you on it.

The gap: why companies want this (and college didn't teach it)

An MBA Finance course teaches you what WACC *means*. It does not teach you that the WACC cell should be a single blue input feeding twenty grey formula cells, or that hard-coding 0.12 inside a formula is a firing-level habit in a model that gets reviewed. College evaluates *answers*; employers evaluate *the workbook that produced the answer* — because someone else has to audit, update, and trust it next quarter.

The specific gaps:

- You were graded on getting a number; industry grades you on a model a stranger can follow.
- You used the mouse; a pro closes books using the keyboard because at 200 rows the mouse is the bottleneck.
- You typed numbers into formulas; industry demands separation of **inputs, calculations, and outputs**.
- You never had to explain *why* a cell shows what it shows — pros use auditing tools (F2, F9, Ctrl+[) to defend every figure.

Closing this gap is the cheapest, highest-return week of study before any finance interview.

What "proficient" looks like

The concrete bar. A job-ready person can, **unaided and mostly without a mouse**:

- Navigate and edit a 5,000-row sheet using only the keyboard (Ctrl+Arrows, Ctrl+Shift+Arrows, Ctrl+PgUp/PgDn).
- Build a workbook with a clear structure: an **Inputs** tab (blue font), **Calc** tabs (black), and an **Output/MIS** tab (formatted for print).
- Format Indian financials: lakhs/crores scaling, `(1,234)` for negatives in red, `-` for zero, ₹ symbol only where needed.
- Explain absolute vs relative refs and fix a broken drag by adding `$` in the right place — using F4, not retyping.

- Create and use named ranges so `=Price*Qty` reads instead of `=B4*C4`.
- Audit any formula: trace precedents (Ctrl+[), evaluate a sub-part with F9, and step through with Evaluate Formula.

If you can do those six without hesitating, you clear the practical filter for most entry finance roles.

Hands-on: how to actually do it

Keyboard-first workflow

Train your hands off the mouse. The core set:

Shortcut	Does
`Ctrl + Arrow`	Jump to edge of data block
`Ctrl + Shift + Arrow`	Select to edge of data block
`Ctrl + Space` / `Shift + Space`	Select column / row
`Alt + =`	AutoSum
`Ctrl + ;`	Insert today's date (static)
`F2`	Edit cell (see the formula)
`F4`	Toggle `$` locks / repeat last action
`Ctrl + [`	Select precedent cells
`Ctrl + \`	Select cells whose value differs across a row
`Alt + E, S, V`	Paste Special → Values
`Ctrl + T`	Convert range to Table
`Ctrl + PgUp / PgDn`	Move between sheets

The **ribbon accelerator** `Alt` is the master key: press `Alt` and Excel prints letters over every command. `Alt, H, 0` deletes a decimal. `Alt, A, S, S` opens Sort. Learn the path once and you never reach for the mouse.

Number formatting for finance (India)

Use **Custom Format** (`Ctrl + 1` → Number → Custom). The four sections are `positive;negative;zero;text`.

```
Standard accounting (red negatives, dash for zero):
#,##0.00;[Red](#,##0.00);"-"

Whole rupees with symbol:
[$₹-en-IN] #,##0;[Red]([$₹-en-IN] #,##0)

Indian lakh/crore digit grouping (12,34,56,789):
[≥10000000]##\,##\,##\,##0;[≥100000]##\,##\,##0;##,##0

Show figures in ₹ lakhs (scale by /100000 using a trailing comma trick won't work in INR;
instead divide in a formula, then format):
#,##0.00" L"
```

Rule pros follow: **never divide by 100000 inside the number format for lakhs** — do the division in a helper cell or state "₹ in lakhs" in the header and divide the source. Keep formatting and math separate.

Absolute / relative references

- `A1` – relative: both move when copied.
- `$A$1` – absolute: locked.
- `A$1` – row locked (mixed).
- `$A1` – column locked (mixed).

Press `F4` while the cursor is on a reference to cycle `A1 → $A$1 → A$1 → $A1`. The classic use: a tax-rate cell you multiply every row against.

```
=E4*$B$1        // B1 holds the GST rate; lock it so every row hits the same cell
```

Named ranges

Select the cell, type a name in the **Name Box** (top-left) or `Ctrl + F3` (Name Manager). Now formulas read like English:

```
=Taxable_Value*GST_Rate
=SUMIFS(Amount, State, "Karnataka")
```

Scope named ranges to the workbook, avoid spaces (use underscores), and never name a range something that looks like a cell (`Q1` is illegal).

Auditing tools

- **Trace Precedents:** `Alt, M, P` (or Formulas ribbon) draws arrows to the cells feeding this one. `Ctrl + [` jumps to them.
- **Trace Dependents:** `Alt, M, D`.
- **F9 inside a formula:** highlight any part of a formula in edit mode and press F9 — Excel replaces it with its live result so you can see *which piece* is wrong. Press `Esc` (not Enter) to avoid overwriting.

- **Evaluate Formula:** `Alt, M, V` steps through calculation order.
- **Show Formulas:** `Ctrl +` `` (grave) toggles the whole sheet between values and formulas — the fastest audit view.

Worked example / mini-project

Build a one-tab **GST output MIS** for a trading firm. Reproduce this exactly.

Inputs (blue font, put the rate in a locked cell `B1`):

	A	B	C	D
1	GST Rate	18%		
3	Invoice	State	Taxable (₹)	GST (₹)
4	INV-001	Karnataka	1,20,000	=C4*$B$1
5	INV-002	Maharashtra	85,000	=C5*$B$1
6	INV-003	Karnataka	2,40,000	=C6*$B$1
7	INV-004	Tamil Nadu	60,000	=C7*$B$1

Now the summary block using **named ranges**. Select `C4:C7`, name it `Taxable`; name `B4:B7` as `State`; name `D4:D7` as `GST`.

```
Total taxable:      =SUM(Taxable)                  → 5,05,000
Total GST:          =SUM(GST)                      → 90,900
Karnataka taxable:  =SUMIFS(Taxable,State,"Karnataka")   → 3,60,000
Invoice count:      =COUNTA(State)                 → 4
Highest invoice:    =MAX(Taxable)                  → 2,40,000
```

Add a lookup so a user can type a state and get its GST:

```
=SUMIFS(GST, State, G1)     // G1 = "Maharashtra" → 15,300
```

Format column C and D with `#,##0;[Red](#,##0)`. Now **audit it**: click the Total GST cell, press `Ctrl + [` — arrows should point only to `D4:D7`. Click a single GST cell, press `F2`, highlight `$B$1`, press `F9` — it shows `0.18`. Press `Esc`. If you change `B1` to 12%, every GST cell and both totals update because you locked the rate instead of hard-coding it. That single behaviour — one input, cascading correctly — is what an interviewer is checking for.

How it's tested

Timed practical (most common for FP&A/MIS/audit): you get a raw dump — say 2,000 rows of sales — and 20–30 minutes to produce a summary. They watch whether you reach for the mouse, whether you use `Ctrl+T` and `SUMIFS`, and whether your negatives are formatted. Speed with keyboard navigation is the visible signal.

Big 4 audit assessment: clean a messy trial balance, tie a subtotal, and flag hard-coded overrides inside formulas. They deliberately plant a cell like `=SUM(B2:B40)+500` for you to catch using audit tools.

Interview questions you should answer instantly:

- "Difference between `A1`, `$A$1`, and `A$1`, and when do you use each?"
- "How do you check what feeds a total?" (Trace Precedents / Ctrl+[)
- "A formula copied down gives wrong answers from row 2 — what's the likely cause?" (missing `$` on a constant reference)
- "How would you present ₹4,50,00,000 in a board deck?" (in ₹ crore, one decimal, labelled)
- "What does F9 do inside a formula?"

Common mistakes & how pros avoid them

Mistake	Why it hurts	The pro habit
Hard-coding numbers in formulas (`=C4*0.18`)	Nobody can find or update the rate	One input cell, referenced everywhere
Forgetting `$` before dragging	Silent wrong totals	F4 to lock, then drag
Using the mouse for everything	Slow; visible in a timed test	Ctrl+Arrow navigation
Merged cells everywhere	Breaks sorting, `SUMIFS`, selection	"Center Across Selection" instead
Dividing by 1,00,000 inside number format	Format ≠ math; breaks totals	Divide in a helper cell, label the header
No colour coding	Reviewer can't tell input from formula	Blue = input, black = formula, green = link
Overwriting after F9	Corrupts the formula	Always Esc, never Enter, after F9
Volatile clutter (`NOW()`, `INDIRECT` everywhere)	Slow, recalcs constantly	Use static `Ctrl+;` dates, minimise volatiles

Learn-it roadmap & resources

Realistic time to the employable bar: **2–3 weeks at 1 hour/day**, if you actually rebuild examples rather than watch.

- **Week 1:** Keyboard navigation + shortcuts. Rule: unplug the mouse for one hour daily. Practise `Ctrl+Arrow`, `Alt` ribbon paths, `F2`, `F4`.
- **Week 2:** Number formatting, absolute refs, named ranges. Rebuild the GST MIS above from scratch three times.
- **Week 3:** Auditing (Trace Precedents, F9, Evaluate Formula) + workbook structure (inputs/calc/output separation, colour coding).

Resources:

- **Free:** ExcelJet shortcut list and "Excel Formula" glossary; Microsoft's own support pages for Custom Number Formats; Corporate Finance Institute's free Excel fundamentals articles.

- **Free video:** Leila Gharani and ExcelIsFun (YouTube) — pick one 30-minute foundations playlist and finish it.
- **Paid/certification: Microsoft Office Specialist: Excel Associate (MO-200)** — globally portable, cheap, and a real line on a CV. CFI's **FMVA** covers this plus modelling (more relevant once you hit Chapter 3+). For India roles, no certificate beats being visibly fast in the timed test — but MO-200 gets you shortlisted.

Practice data: export any bank statement or a GSTR-2B to Excel and force yourself to summarise it keyboard-only.

Quick-reference

Navigation & editing

```
Ctrl+Arrow          jump to data edge
Ctrl+Shift+Arrow    select to edge
Ctrl+T              make a Table
Alt+=               AutoSum
Ctrl+;              static date
F2                  edit / see formula
F4                  toggle $ locks
Alt,E,S,V           Paste values
Ctrl+`              show all formulas
```

References

```
A1     relative      $A$1   fully locked
A$1    row locked    $A1    column locked
F4     cycles all four states
```

Auditing

```
Ctrl+[        select precedents
Alt,M,P       trace precedents (arrows)
Alt,M,D       trace dependents
F9            evaluate selected formula part (then Esc)
Alt,M,V       Evaluate Formula step-through
```

Finance number formats (Ctrl+1 → Custom)

```
#,##0.00;[Red](#,##0.00);"-"        accounting, red negatives
[$₹-en-IN] #,##0                     rupee symbol
#,##0.00" L"                         label as lakhs (divide source separately)
```

Golden rules: one input cell per assumption · lock rates with `$` · blue inputs / black formulas · never hard-code · Esc after F9 · label ₹ scale in the header.

Essential functions every analyst must know

What it is & where it's used

This is the working vocabulary of a finance analyst. Ninety percent of real Excel work in accounts, FP&A, audit, tax, and treasury is **conditional aggregation** (sum/count/average by criteria), **decision logic** (if this then that), **date arithmetic** (interest periods, ageing, GST tax periods, TDS due dates), **text surgery** (cleaning ERP exports, splitting GSTINs, formatting reports), and **error control** (so one bad cell doesn't blow up a model).

Where you'll use each one daily:

Function group	Real job task
SUMIFS / COUNTIFS / AVERAGEIFS	Ledger totals by cost centre, sales by region, avg invoice value by customer
IF / IFS / AND / OR	Credit-limit checks, TDS-applicability flags, commission slabs
EOMONTH / EDATE / YEARFRAC	GST return periods, EMI schedules, depreciation, receivables ageing
LEFT / MID / TEXT / TEXTSPLIT	Splitting GSTIN, cleaning Tally/SAP dumps, building report labels
IFERROR	Bulletproofing lookups and divisions in a live model

Roles that live in these functions: **Accounts Executive, AP/AR analyst, FP&A analyst, Statutory/Internal Audit associate, Tax (Direct + GST) associate, Treasury/MIS analyst, Investment Banking/PE junior.**

The gap: why companies want this (and college didn't teach it)

An MBA or CA Inter teaches you *what* a variance is, *what* receivables ageing means, and *how* GST periods work — conceptually. It almost never sits you in front of a 40,000-row `.csv` dump from Tally/SAP and says "give me month-wise sales by state, net of returns, by 3 PM." That translation layer — concept to a formula that runs on messy real data — is exactly the paid skill.

The specific gaps employers see in fresh hires: - They reach for `SUMIF` (single condition) and get stuck the moment there are two conditions. **`SUMIFS` is the real-world default.** - They nest five `IF`s into an unreadable monster instead of `IFS` or a lookup. - They subtract dates manually and get ageing buckets wrong across month-ends. - They retype data instead of using `TEXT` / `TEXTSPLIT` to reshape an export in seconds. - Their models show `#DIV/0!` and `#N/A` in front of a manager — an instant credibility hit that `IFERROR` prevents.

What "proficient" looks like

The concrete bar. A job-ready person can, **unaided and in minutes**:

1. Write a multi-condition `SUMIFS` / `COUNTIFS` / `AVERAGEIFS` including a **date range** and a **wildcard** criterion.
2. Replace nested `IF`s with `IFS`, and combine conditions with `AND` / `OR`.
3. Build a **receivables ageing** with `EOMONTH` / `TODAY` / `YEARFRAC` without hardcoding dates.
4. Split a GSTIN or clean a "LASTNAME, Firstname" export using `LEFT` / `MID` / `TEXTSPLIT`.

5. Wrap risky formulas in `IFERROR` so the model never shows a raw error.
6. Explain **why** `SUMIFS` (sum-range first) argument order differs from `SUMIF` (range, criteria, sum-range) — a classic interview trap.

Hands-on: how to actually do it

Assume a sales table: `A:Date, B:State, C:Customer, D:Product, E:Amount, F:Type` (Sale/Return).

Conditional aggregation

```
' Total SALE amount for Maharashtra
=SUMIFS(E:E, B:B, "Maharashtra", F:F, "Sale")

' Sales in a date window (Q1 FY26: 1-Apr to 30-Jun-2025)
=SUMIFS(E:E, A:A, "≥"&DATE(2025,4,1), A:A, "≤"&DATE(2025,6,30))

' Wildcard: all customers whose name starts with "Reliance"
=SUMIFS(E:E, C:C, "Reliance*")

' Count of return invoices; average invoice value for a product
=COUNTIFS(F:F, "Return")
=AVERAGEIFS(E:E, D:D, "Widget-A", F:F, "Sale")
```

Note the order: `SUMIFS(sum_range, criteria_range1, criteria1, …)`. Join operators to a cell with `&`: `A:A, "≥"&G1`.

Decision logic

```
' Simple flag: is TDS applicable? (single-vendor limit Rs 30,000 u/s 194J)
=IF(E2>30000, "Deduct TDS", "No TDS")

' Multi-slab commission with IFS (no nesting)
=IFS(E2≥1000000,"5%", E2≥500000,"3%", E2≥100000,"1%", TRUE,"0%")

' AND / OR
=IF(AND(F2="Sale", E2>500000), "High-value sale", "Normal")
=IF(OR(B2="J&K", B2="Ladakh"), "Special zone", "Regular")
```

Date functions

```
' GST return period end (last day of invoice month)
=EOMONTH(A2, 0)                 ' 15-May-25  → 31-May-25

' Credit due date = invoice date + 45 days; or +2 months
=A2+45
=EDATE(A2, 2)                   ' same day, two months later

' EMI / depreciation date one month out
=EDATE(A2, 1)

' Receivables ageing in days and in years
=TODAY()-A2                                  ' days outstanding
=YEARFRAC(A2, TODAY(), 1)                     ' fraction of a year (actual/actual)

' Ageing bucket
=IFS(TODAY()-A2≤30,"0-30", TODAY()-A2≤60,"31-60",
     TODAY()-A2≤90,"61-90", TRUE,"90+")
```

Text functions (cleaning exports)

```
' GSTIN = 27ABCDE1234F1Z5 in cell H2
=LEFT(H2,2)                     ' 27  → state code
=MID(H2,3,10)                   ' ABCDE1234F → PAN embedded in GSTIN
=MID(H2,3,10)                   ' also the PAN for TDS matching

' Split "MUMBAI, Maharashtra" into two cells (Excel 365 / dynamic array)
=TEXTSPLIT(I2, ", ")            ' spills MUMBAI | Maharashtra

' Build a clean label / format a number as text
=TEXT(A2,"mmm-yyyy")            ' May-2025
=TEXT(E2,"#,##0")              ' 12,34,567  (use "[≥10000000]#,##,##,##0" for full Indian gro
=D2&" ("&TEXT(E2,"Rs #,##0")&")"   ' Widget-A (Rs 5,00,000)
```

Error control

```
' Never show #N/A / #DIV/0! to a manager
=IFERROR(VLOOKUP(C2, Master!A:D, 4, FALSE), "Not found")
=IFERROR(E2/G2, 0)              ' safe division → 0 instead of #DIV/0!
```

Worked example / mini-project

Build a one-screen receivables + sales MIS. Data: a debtors export with `A:Invoice No, B:Customer, C:Invoice Date, D:GSTIN, E:Amount, F:State`. 12 rows, realistic:

Invoice	Customer	Inv Date	GSTIN	Amount (Rs)	State
INV001	Reliance Retail	05-Feb-2025	27AAACR...	4,50,000	Maharashtra
INV002	Tata Steel	20-Mar-2025	24AAACT...	8,20,000	Gujarat
INV003	Infosys Ltd	12-May-2025	29AAACI...	2,10,000	Karnataka

Now build these output cells (today assumed **03-Jul-2025**):

```
' 1. State code from GSTIN (audit check vs State column)
=LEFT(D2,2)                                    ' 27 → should map to Maharashtra

' 2. Days outstanding + ageing bucket
=TODAY()-C2
=IFS(TODAY()-C2≤30,"0-30", TODAY()-C2≤60,"31-60",
     TODAY()-C2≤90,"61-90", TRUE,"90+")        ' INV001 → 90+

' 3. Total outstanding over 90 days (the number a CFO asks for first)
=SUMIFS(E:E, C:C, "<"&(TODAY()-90))            ' all invoices older than 90 days

' 4. Sales by state (feeds a state-wise summary)
=SUMIFS($E$2:$E$13, $F$2:$F$13, "Maharashtra")

' 5. Count of overdue accounts + safe average ticket
=COUNTIFS($C$2:$C$13, "<"&(TODAY()-60))
=IFERROR(AVERAGEIFS($E$2:$E$13,$F$2:$F$13,"Gujarat"),0)

' 6. Report label
="Overdue >90d: "&TEXT(SUMIFS(E:E,C:C,"<"&(TODAY()-90)),"Rs #,##,##0")
```

Reproduce it: paste 12 rows, drop these formulas, and you have a live dashboard that recalculates every time you refresh the export. That is a deliverable you can show in a first-week task.

How it's tested

Interview questions (verbal): - "Difference between `SUMIF` and `SUMIFS`?" (answer: multiple criteria; and the sum-range moves to the *first* argument). - "How do you sum only rows in a date range?" (criteria with `"≥"&start` and `"≤"&end`). - "How do you avoid `#N/A` in a lookup?" (`IFERROR` / `IFNA`). - "Why not just nest `IF`s?" (readability, `IFS`, or a lookup table).

Practical / assessment tests (the real filter): - A **timed 30-45 min Excel test**: a raw sheet + 6-10 tasks — "total sales for South region in Q2", "flag invoices over Rs 1,00,000 needing approval", "extract PAN from GSTIN", "age these receivables". No internet, formulas graded on correctness *and* whether they use absolute refs / dynamic dates. - A **"clean this export"** task: a `LASTNAME, First` or merged-column dump you must split with `TEXTSPLIT` / `MID`. - Case rounds in FP&A/IB often hand you a messy CSV and expect a summary table built with `SUMIFS` + a pivot within 20 minutes.

Pass signal: you finish, formulas are robust (no hardcoded dates, wrapped in `IFERROR`), and you can explain each one.

Common mistakes & how pros avoid them

Mistake	Fix / pro habit
Wrong `SUMIFS` argument order (sum-range last)	Sum-range is **first**; muscle-memory it
Hardcoding `"01-04-2025"` as text criteria	Use `"≥"&DATE(2025,4,1)` — real dates, comparable
Full-column refs `E:E` in a huge model → slow	Use fixed ranges `$E$2:$E$50000` in big files
Nesting 6 `IF`s	Use `IFS`, or a lookup table with `XLOOKUP`
Manual date subtraction across month-end	`EOMONTH` / `EDATE`, never add "30" for a month
`LEFT/MID` on inconsistent-length text	Anchor with `FIND` / `SEARCH` or use `TEXTSPLIT` on a delimiter
`IFERROR` hiding a *real* bug	Only wrap known-safe risks (div-by-zero, missing lookup), not everything
Locking references wrong when dragging	`F4` to toggle `$`; lock criteria ranges, not the criteria cell

Learn-it roadmap & resources

Time to proficiency: 2-3 weeks of daily practice (1 hr/day) to clear a timed test; ~2 months to be genuinely fast on messy data.

Week	Focus
1	`SUMIFS/COUNTIFS/AVERAGEIFS`, absolute refs, date criteria
2	`IF/IFS/AND/OR`, `EOMONTH/EDATE/YEARFRAC`, ageing project
3	Text functions on real ERP dumps, `IFERROR`, build the MIS above

Resources - Free: **ExcelJet** (function reference + examples), **Chandoo.org**, **Corporate Finance Institute (CFI)** free Excel course, Microsoft's own function docs. - Paid: CFI's **FMVA** certification (Excel-heavy, finance-specific), **Wall Street Prep**, Udemy "Microsoft Excel — Excel from Beginner to Advanced" (Kyle Pew). - Practice data: export your own Tally/any sample GST invoice register and re-solve the mini-project. - Certification worth naming on a CV: **Microsoft Office Specialist (MOS) Excel Associate/Expert**, or **FMVA** for finance roles.

Quick-reference

```
' AGGREGATION
=SUMIFS(sum_rng, crit_rng1, crit1, crit_rng2, crit2)
=COUNTIFS(crit_rng1, crit1, ...)
=AVERAGEIFS(avg_rng, crit_rng1, crit1, ...)
=SUMIFS(E:E, A:A, "≥"&DATE(2025,4,1), A:A, "≤"&DATE(2025,6,30))   ' date window
=SUMIFS(E:E, C:C, "Reliance*")                                    ' wildcard

' LOGIC
=IF(test, if_true, if_false)
=IFS(cond1,val1, cond2,val2, TRUE,default)
=AND(c1,c2)   =OR(c1,c2)

' DATES
=EOMONTH(date, months)     ' last day of month, offset
=EDATE(date, months)       ' same day, months out
=YEARFRAC(start, end, 1)   ' year fraction (basis 1 = actual/actual)
=TODAY()-date              ' days outstanding

' TEXT
=LEFT(txt,n)   =RIGHT(txt,n)   =MID(txt,start,n)
=TEXTSPLIT(txt, ", ")      ' split on delimiter (Excel 365)
=TEXT(value,"mmm-yyyy")    ' or "#,##0" / "Rs #,##,##0"

' ERROR CONTROL
=IFERROR(formula, fallback)
=IFNA(VLOOKUP(...), "Not found")
```

GSTIN cheat: `LEFT(gstin,2)` = state code, `MID(gstin,3,10)` = PAN. **Argument order trap:** `SUMIF(range, criteria, sum_range)` but `SUMIFS(sum_range, range, criteria)` — sum-range **moves to front** in the plural version.

Lookup & Reference Mastery

What it is & where it's used

Lookup functions answer one question that every finance job asks a hundred times a day: "*Given this ID, fetch me that value.*" Given a vendor code, fetch the GSTIN. Given an employee ID, fetch the CTC. Given a ledger name, fetch the closing balance. Given an ISIN, fetch the market price.

You have a "lookup table" (a master somewhere) and a "working sheet" (your model), and you need to pull data from one into the other by a matching key. That is 60-70% of real analyst work — you rarely type numbers, you *stitch datasets together*.

Where it shows up by role:

Role	Typical lookup job
Accounts payable	Match invoice numbers to a payment register; pull vendor bank details
GST / Tax	Reconcile GSTR-2B purchases against your purchase register (match by invoice + GSTIN)
FP&A / Analyst	Map trial-balance ledgers to P&L line items; build a driver-based model
Audit	Vouch a sample — pull supporting entries by voucher number
Treasury	Fetch FX rates or interest rates by date/currency
Investment / Equity research	Pull financials by ticker across years

If you can only do one Excel thing well, make it lookups. It is the single most tested skill in a finance Excel screen.

The gap: why companies want this (and college didn't teach it)

MBA and CA coursework teaches you *what* a reconciliation is, or *why* you allocate overheads — the concepts. It almost never makes you sit with two messy exports and physically join them. So freshers arrive knowing the theory of a bank reconciliation but not knowing how to match 4,000 bank lines to 4,000 book lines in ninety seconds.

The specific gaps employers see:

- **Manual matching mindset.** Freshers filter, eyeball, and copy-paste. A 2,000-row reconciliation takes them a day; a pro does it in five minutes with a lookup and a `MATCH`-based flag.
- **VLOOKUP-only, and fragile.** Many candidates know only `VLOOKUP`, which breaks the moment someone inserts a column and cannot look leftward. They have never met `INDEX-MATCH` or `XLOOKUP`.
- **No handling of "not found."** Real data has unmatched rows. Untrained users get `#N/A` everywhere and freeze. Pros trap it with a default and turn `#N/A` into a *finding* ("these 12 invoices are in books but not in 2B").
- **No dynamic arrays.** `FILTER`, `UNIQUE`, `SORT` (Excel 365 / 2021+) let you build a live sub-report with one formula. Colleges teach none of this because their labs run Excel 2007.

Closing this gap is what turns a "knows Excel" line on your CV into "does the reconciliation unaided by day two."

What "proficient" looks like

A job-ready person can, unaided:

- Choose the right tool: `XLOOKUP` for a single fetch, `INDEX-MATCH` for legacy files, `FILTER` when they need *many* matching rows, not one.
- Do a **two-way lookup** (row key × column key) — e.g., pull the March figure for "Salaries" from a month × ledger grid.
- Handle not-found gracefully and treat unmatched rows as an audit output.
- Do an **approximate/range lookup** — slab-based, e.g., income-tax slabs or a commission grid.
- Lock references correctly (`$`) so a formula fills down and across without drifting.
- Explain *why* `XLOOKUP` beats `VLOOKUP` and rewrite an old `VLOOKUP` model without breaking it.

Hands-on: how to actually do it

1. XLOOKUP — the modern default

Syntax: `=XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])`

```
Pull a vendor's GSTIN by vendor code:
=XLOOKUP(A2, Vendors[Code], Vendors[GSTIN], "Not found")

Pull the closing balance for a ledger:
=XLOOKUP(A2, Ledger[Name], Ledger[Closing], 0)
```

- `if_not_found` = `"Not found"` — no more `#N/A`, no `IFERROR` wrapper.
- Return array can be **left of** the lookup array. VLOOKUP cannot.
- Return a whole row by pointing `return_array` at multiple columns: `=XLOOKUP(A2, Ledger[Name], Ledger[Opening]:Ledger[Closing])` spills opening→closing across.

2. INDEX-MATCH — works in every Excel version

```
=INDEX(Ledger[Closing], MATCH(A2, Ledger[Name], 0))
```

`MATCH` finds the *position* of the key; `INDEX` returns the value at that position. The `0` means exact match. Use this in any file that must open in Excel 2016 or older, or where colleagues don't have 365.

3. Two-way lookup (row × column)

Grid: ledgers down column A, months across row 1, figures inside.

```
XLOOKUP nested — fetch Salaries for Mar:
=XLOOKUP(G2, A2:A50, XLOOKUP(H2, B1:M1, B2:M50))

INDEX with two MATCHes — the classic:
=INDEX(B2:M50, MATCH(G2, A2:A50, 0), MATCH(H2, B1:M1, 0))
```

`G2` = "Salaries", `H2` = "Mar". The inner lookup picks the column; the outer picks the row.

4. Approximate / slab lookup (tax, commission)

Set a slab table sorted ascending by lower bound. Use `match_mode = -1` (exact or next smaller):

```
Slab table  A:B  →  lower bound | rate
=XLOOKUP(Income, SlabLower, SlabRate, , -1)
```

Old-style equivalent: `=VLOOKUP(Income, SlabTable, 2, TRUE)` — the `TRUE` is the range match.

5. Dynamic arrays — FILTER, UNIQUE, SORT (365 / 2021+)

```
Every invoice for one vendor (many rows, one formula):
=FILTER(Data, Data[Vendor]="ABC Traders", "None")

Unmatched invoices (in books, not in 2B):
=FILTER(Books[Inv], ISNA(XLOOKUP(Books[Inv], GSTR2B[Inv], GSTR2B[Inv])), "All matched")

Distinct list of ledgers:
=UNIQUE(Data[Ledger])

Sorted, de-duped vendor master:
=SORT(UNIQUE(Data[Vendor]))

Top 5 debtors by amount:
=SORT(FILTER(Data, Data[Type]="Debtor"), 3, -1)
```

These **spill** — one formula, a whole live block that recalculates as data changes. This is how you build a self-updating exception report.

Worked example / mini-project: GSTR-2B vs Purchase Register reconciliation

You have two exports. Match input tax credit (ITC) claimed in books against what shows in GSTR-2B.

Purchase Register (sheet `PR`), columns: `A` Invoice No, `B` GSTIN, `C` Taxable, `D` GST. **GSTR-2B** (sheet `2B`), columns: `A` Invoice No, `B` GSTIN, `C` Taxable, `D` GST.

Sample data:

PR Invoice	GSTIN	PR GST (₹)
INV-101	27AABCU9603R1ZM	18,000
INV-102	29AAACI1234K1Z5	9,000
INV-103	27AABCU9603R1ZM	4,500

Step 1 — flag whether each PR invoice is in 2B, in column `E` of `PR`:

```
=IF(ISNA(XLOOKUP(A2, '2B'!A:A, '2B'!A:A)), "Missing in 2B", "Matched")
```

Step 2 — compare the GST amount (catch value mismatches), column `F`:

```
=IFERROR(D2 - XLOOKUP(A2, '2B'!A:A, '2B'!D:D), "Not in 2B")
```

A non-zero, non-error result = the same invoice with a different tax figure — a real reconciliation exception.

Step 3 — build the exception report on a fresh sheet, one formula:

```
=FILTER(PR!A2:F100, PR!E2:E100="Missing in 2B", "All reconciled")
```

Step 4 — total ITC at risk (in books but not in 2B, i.e., not claimable this month):

```
=SUMIF(PR!E:E, "Missing in 2B", PR!D:D)
```

Result: say ₹9,000 (INV-102) is missing in 2B → that ITC is provisionally not available, flag to the vendor. You just did in four formulas what a fresher does in half a day of filtering. Reverse the direction (2B not in PR) to catch invoices you forgot to book.

How it's tested

The practical test (most common). You get two raw exports and 20-45 minutes: "*Reconcile these and tell me the unmatched items and the total difference.*" They watch whether you reach for a lookup or start filtering manually. Reaching for `XLOOKUP` / `INDEX-MATCH` + a `FILTER` exception list is an instant pass.

Live Excel screen. Screen-share, a small table, and prompts like: "Pull the salary for employee E-045." "Now the master got a column inserted — did your formula survive?" (Tests whether you hard-coded a column index in `VLOOKUP`.)

Interview questions you should be able to answer cold:

- Difference between `VLOOKUP` and `INDEX-MATCH`? (Left-lookup, column-insert safety, speed.)
- Why does `VLOOKUP` return the wrong value sometimes? (Default `TRUE` approximate match on unsorted data.)
- How do you look up to the left? (`INDEX-MATCH` or `XLOOKUP`.)
- How do you return multiple matching rows? (`FILTER`, not any single-cell lookup.)
- What does the 4th argument of `MATCH` / `XLOOKUP` do? (Match mode: exact vs next-smaller for slabs.)

Common mistakes & how pros avoid them

Mistake	What breaks	Pro fix
`VLOOKUP` with `TRUE` (or omitted 4th arg)	Silent wrong values on unsorted data	Always pass `0` / `FALSE` for exact; use `-1` deliberately for slabs
Hard-coded column index (`VLOOKUP( ... ,4,0)`)	Breaks when a column is inserted	`XLOOKUP` / `INDEX-MATCH` reference the column by name/range
Forgetting `$` locks	Formula drifts when filled down	Lock the lookup array: `XLOOKUP(A2,$D$2:$D$99,$E$2:$E$99)` — or use Tables
Wrapping everything in `IFERROR`	Hides *real* errors, not just `#N/A`	Use `XLOOKUP` 's `if_not_found` , or `IFNA` (traps only `#N/A`)
Lookup key type mismatch	"INV-101" text vs 101 number → `#N/A`	Standardise with `TRIM` , `TEXT` , or `VALUE` ; watch trailing spaces
Duplicate keys	Lookup returns only the first match	De-dupe with `UNIQUE` , or `FILTER` to see all; add a composite key
Composite matching (invoice **and** GSTIN)	Same invoice no. across vendors matches wrong row	Concatenate a key: `A2&"|"&B2` in both sheets, then look up that
Volatile giant ranges (`A:A`) on 100k rows	Workbook crawls	Reference actual used range or a Table

Learn-it roadmap & resources

Realistic timeline: 2-3 weeks of daily practice to interview-ready; solid in a couple of months of real use.

- **Days 1-3:** `XLOOKUP` and `INDEX-MATCH` exact matches; absolute references; `IFNA` .
- **Days 4-7:** Two-way lookups; approximate/slab lookups; convert an old `VLOOKUP` file to `XLOOKUP` .
- **Week 2:** Dynamic arrays — `FILTER` , `UNIQUE` , `SORT` ; build the GSTR-2B recon above end to end.
- **Week 3:** Composite keys, duplicate handling, and doing a full reconciliation timed under 20 minutes.

Resources

- Microsoft's own `XLOOKUP` / `FILTER` support pages (free, authoritative — search "XLOOKUP function Microsoft").
- ExcelJet (free formula reference with examples) — exceljet.net.
- Chandoo.org and Leila Gharani (YouTube) — free, finance-flavoured.
- Paid, if you want a certificate: **Microsoft Office Specialist: Excel Associate (MO-200)** or **Expert (MO-201)** — recognised, exam-based, ~₹4,000-5,000. For finance modelling context, a Corporate Finance Institute (CFI) or Wall Street Prep Excel module.
- Practice data: export any GSTR-2B JSON from the GST portal and your own purchase register — real data beats toy datasets.

Quick-reference

```
XLOOKUP    =XLOOKUP(key, lookup_col, return_col, "not found", match_mode, search_mode)
INDEX-MATCH =INDEX(return_col, MATCH(key, lookup_col, 0))
Two-way     =INDEX(grid, MATCH(rowkey,rows,0), MATCH(colkey,cols,0))
Slab/range  =XLOOKUP(val, lower_bounds, rates, , -1)   'exact-or-next-smaller
Left lookup =XLOOKUP or INDEX-MATCH  (VLOOKUP cannot)
Many rows   =FILTER(data, criteria_range=criteria, "none")
Distinct    =UNIQUE(range)      Sorted =SORT(range, col, -1)
Composite   key = A2&"|"&B2  in both sheets, then look up the key
Trap N/A    =IFNA(formula, "-")   'only #N/A, not other errors
```

VLOOKUP	XLOOKUP	Winner
Looks only rightward	Looks either direction	XLOOKUP
Hard-coded column index breaks on insert	Column referenced directly	XLOOKUP
Default approximate match (dangerous)	Default exact match	XLOOKUP
`#N/A` needs `IFERROR` wrapper	Built-in `if_not_found`	XLOOKUP
Works in all versions	Excel 365 / 2021+ only	VLOOKUP (legacy only)

Match modes (XLOOKUP/MATCH): `0` exact · `-1` exact or next smaller (slabs, ascending) · `1` exact or next larger · `2` wildcard.

Power Query: Cleaning Real Company Data

What it is & where it's used

Power Query is the built-in **ETL engine** (Extract, Transform, Load) inside Excel and Power BI. You point it at messy source data — a bank statement export, a GST portal download, ten branch files in a folder, a Tally ledger dump — record a sequence of cleaning steps, and it produces a clean table. The magic word is **refreshable**: when next month's file arrives, you drop it in and hit **Refresh**. All your steps replay automatically. No re-cleaning, no re-formulas.

Find it in Excel under **Data → Get Data** (Windows Excel 2016+ and Microsoft 365; the ribbon group is called "Get & Transform Data"). It writes its logic in a language called **M**, but 90% of the job is done with clicks in the Power Query Editor.

Roles that live in Power Query: **FP&A analysts** consolidating branch actuals, **accounts payable/receivable teams** reconciling ledgers, **GST/tax executives** cleaning portal downloads for GSTR-2B vs purchase-register matching, **MIS analysts** building monthly management packs, and anyone who currently spends the first three days of every month copy-pasting and running find-replace.

The gap: why companies want this (and college didn't teach it)

Your MBA taught you ratio analysis on data that was **already clean**. Real company data never is. The gap employers pay to close:

- **Source data is filthy.** Merged cells, ₹ signs stored as text, dates as "01-Apr-25" strings, blank header rows, totals mixed into detail rows, "1,20,000" with Indian comma grouping that Excel reads as text.
- **The same clean-up runs every single month.** College projects are one-shot. A job is the *same* report 12 times a year. Manual cleaning that "only takes 2 hours" costs 24 hours a year and breaks the moment a colleague is on leave.
- **VLOOKUP-and-paste doesn't scale.** When AP hands you a 40,000-row purchase register and 38,000-row GSTR-2B, a manual match is impossible. Power Query merges them in seconds.

Employers have watched analysts burn days on this. Someone who says "give me the raw dump, I'll build a refreshable query" is instantly more valuable than someone who reaches for a fresh round of copy-paste.

What "proficient" looks like

A job-ready person can, unaided:

1. **Import** from Excel, CSV, a whole folder, and a database — and know that **Get Data → From Folder** stacks 12 monthly files into one table.
2. **Unpivot** a wide "months across columns" report into a tidy long table (the single most-tested skill).
3. **Merge** two queries (Left Outer, Inner, Left Anti) — and know *which* join to use for a reconciliation.
4. **Append** queries to stack same-shape tables.
5. **Remove errors, fill down, split columns, change types, trim/clean text, and replace values** without breaking on next refresh.

6. Understand that steps are **recorded and replayed**, so column names and types must be handled deliberately.

Hands-on: how to actually do it

Import from a folder (consolidate 12 branch files)

```
Data → Get Data → From File → From Folder → pick folder → Combine & Transform
```

Power Query samples the first file, applies the transform to all, and stacks them. Add a step to keep the source file name so you know which branch a row came from — it's the `Source.Name` column.

Unpivot (the interview favourite)

Raw MIS often looks like this:

Branch	Apr	May	Jun
Mumbai	120000	135000	128000
Pune	90000	88000	94000

Select the **Branch** column → right-click → **Unpivot Other Columns**. Result:

Branch	Attribute	Value
Mumbai	Apr	120000
Mumbai	May	135000

Rename `Attribute` → `Month`, `Value` → `Sales`. Now it's a proper table you can pivot, filter, or feed to Power Pivot. **Always use "Unpivot Other Columns"** so new month columns next quarter are captured automatically.

Clean Indian-format numbers stored as text

"1,20,000" imports as text. In the Editor, **Transform** → **Format** → **Trim & Clean**, then **Replace Values**: replace `,` with nothing, then **Change Type** → **Whole Number**. The equivalent M step:

```
= Table.ReplaceValue(Source, ",", "", Replacer.ReplaceText, {"Amount"})
```

Split columns

A cell like `27AABCU9603R1ZM - Uma Traders` (GSTIN + name):

```
Split Column → By Delimiter → " - " → at each occurrence
```

Or split GSTIN to grab the **state code** (first 2 chars): **Split Column** → **By Number of Characters** → **2** → **Once, as far left as possible**. State code `27` = Maharashtra.

Remove errors and fill down

After a type change, bad rows show `Error`. Select the column → **Remove Errors** (or **Replace Errors** with `0`). For a report where the branch name appears once then blanks below it: select column → **Transform** → **Fill** → **Down**.

Merge queries (reconciliation)

To find purchases in your register that are **missing** from GSTR-2B:

```
Home → Merge Queries → pick PurchaseRegister & GSTR2B
→ match on [GSTIN] + [Invoice No]  (Ctrl-click both keys in each table)
→ Join Kind: Left Anti (rows only in first)
```

Join kinds you must know:

Join Kind	Returns	Use for
Left Outer	All left + matches	Enrich data (add vendor name)
Inner	Only matched rows	Confirmed matches
Left Anti	Left rows with NO match	**Missing / unreconciled items**
Right Anti	Right rows with NO match	In 2B but not in books

Append queries (stack)

```
Home → Append Queries → Three or more tables → add Q1, Q2, Q3, Q4
```

Same columns, stacked into one — a full-year table from four quarterly files.

Worked example / mini-project: GSTR-2B vs Purchase Register match

Goal: find input tax credit (ITC) you can claim, and mismatches to chase.

Source 1 — `PurchaseRegister.xlsx` (your books):

GSTIN	Invoice No	Taxable	IGST
27AABCU9603R1ZM	INV-001	100000	18000
29AAGCB1286Q1ZP	INV-002	50000	9000
07AABCU9603R1ZX	INV-003	20000	3600

Source 2 — `GSTR2B.csv` (portal download):

GSTIN	Invoice No	IGST_2B
27AABCU9603R1ZM	INV-001	18000
29AAGCB1286Q1ZP	INV-002	8500

Steps:

1. **Get Data** → **From Excel** → load `PurchaseRegister` as a connection-only query. Repeat **From Text/CSV** for `GSTR2B`.
2. In each, **Trim** the Invoice No (portal data has trailing spaces — a classic silent mismatch killer) and set types.
3. On `PurchaseRegister`: **Merge Queries** → `GSTR2B`, match on `GSTIN` + `Invoice No`, **Left Outer**. Expand `IGST_2B`.
4. Add a **Custom Column** to flag status:

```
= if [IGST_2B] = null then "Missing in 2B"
  else if [IGST] <> [IGST_2B] then "Tax mismatch"
  else "Matched"
```

Result:

Invoice No	IGST	IGST_2B	Status
INV-001	18000	18000	Matched
INV-002	9000	8500	Tax mismatch
INV-003	3600	(null)	Missing in 2B

Close & Load to a sheet. INV-002 has a ₹500 mismatch to query with the vendor; INV-003's ₹3,600 ITC can't be claimed until the vendor files. Next month, drop in the new files and **Refresh** — the whole reconciliation rebuilds in seconds.

How it's tested

In interviews (verbal): - "You have sales in columns Jan–Dec, one row per product. How do you get one row per product-month?" → *Unpivot Other Columns.* - "Which join finds invoices in your books but not on the GST portal?" → *Left Anti.* - "Difference between Merge and Append?" → *Merge = join sideways on a key; Append = stack rows.* - "How do you consolidate 15 branch files with one click each month?" → *From Folder → Combine & Transform.*

Practical test (very common): you're given a deliberately messy workbook — merged cells, ₹ text amounts, months across columns, a second sheet to reconcile against — and 30–45 minutes to produce a clean, refreshable output. They then **change the data and hit Refresh** to check your query didn't hard-code anything.

Red flag they watch for: did you clean it with manual find-replace on the sheet (breaks on refresh) or inside Power Query (survives refresh)? The whole point is the second one.

Common mistakes & how pros avoid them

Mistake	Consequence	Pro fix
Hard-coding a "Changed Type" with fixed column names	Breaks when a column is renamed upstream	Delete unneeded auto-type steps; type only what you need, late
Using "Unpivot Columns" instead of "Unpivot **Other** Columns"	New month columns get dropped next quarter	Always unpivot *other* columns
Not trimming text before a merge	Silent non-matches from trailing spaces	Trim + Clean both key columns first
Filtering by a specific value (e.g. keep only "Apr")	Wrong data after refresh	Filter on logic, not a snapshot value
Loading giant tables to a sheet you don't need	Slow, bloated file	Use **Connection Only** + load to Data Model
Renaming columns then referencing old names later	Step errors on refresh	Rename once, early, and keep it consistent

Learn-it roadmap & resources

Time to proficiency: about 2–3 weeks of evening practice for someone comfortable in Excel. Power Query is unusually learnable because it's click-driven — you can be productive before you understand M.

- **Week 1:** Import (Excel/CSV/Folder), change types, remove columns, split, trim, replace values, fill down.
- **Week 2:** Unpivot, group by, merge (all join kinds), append.
- **Week 3:** Custom columns, conditional columns, a light touch of M, and building one real refreshable monthly report end-to-end.

Resources: - **Microsoft Learn — "Get & Transform in Excel"** (free, official). - **Excel Is Fun** and **MyOnlineTrainingHub** (Mynda Treacy) — free YouTube, finance-friendly examples. - **Book:** *Master Your Data with Excel and Power BI* — Ken Puls & Miguel Escobar (the definitive Power Query reference). - **Certification:** it's covered inside the **Microsoft PL-300 (Power BI Data Analyst)** exam — worth listing on a CV, though for most finance jobs a demonstrated project matters more than the cert.

Quick-reference

Task	Click-path
Import one file	Data → Get Data → From File → From Workbook/CSV
Stack many files	Get Data → From File → From Folder → Combine & Transform
Wide → long	Select key cols → right-click → Unpivot Other Columns
Fill blanks below	Column → Transform → Fill → Down
Clean text	Transform → Format → Trim, then Clean
Split GSTIN state code	Split Column → By Number of Characters → 2
Reconcile / find missing	Home → Merge Queries → Join Kind: **Left Anti**
Enrich (add name)	Merge → **Left Outer** → Expand
Stack same-shape tables	Home → Append Queries
Don't load to sheet	Close & Load To → **Connection Only**
Re-run everything	Data → **Refresh All**

Join cheat: Left Outer = enrich · Inner = confirmed matches · Left Anti = missing in second · Right Anti = extra in second.

Golden rule: never fix data on the sheet — fix it in Power Query, so next month is one click: **Refresh**.

PivotTables & Dashboards

What it is & where it's used

A **PivotTable** is Excel's engine for turning a flat list of transactions into a summary you can slice any way you want — total sales by region, expense by cost centre, GST payable by month — without writing a single formula. A **dashboard** is a one-page visual layer built on top of those pivots: a few charts, some KPI tiles, slicers to filter, all refreshing from one data source.

This is the single most-used Excel skill in day-to-day finance work. Where it shows up:

- **FP&A / MIS analysts** — the monthly MIS pack is 90% pivots and dashboards.
- **Accounts / audit** — reconciling a Tally trial-balance dump, ageing debtors, ledger scrutiny.
- **Tax** — reconciling GSTR-2B against your purchase register (both are flat CSV exports → pivot both → compare).
- **Treasury / management reporting** — cash position, DSO/DPO trends.

If a job description says "prepare monthly MIS," "reporting," or "management dashboards," it means PivotTables. Full stop.

The gap: why companies want this (and college didn't teach it)

MBA and CA syllabi teach you what a *variance* is and how to read a *P&L*. They almost never make you sit with 40,000 rows of raw ledger data and produce a manager-ready one-pager by 10 a.m. That translation step — **raw data → decision-ready summary** — is the actual job, and it's the gap.

Specifically, college leaves you weak on:

College teaches	The job needs
Interpreting a finished P&L	*Building* it from a Tally/ERP dump
SUM / VLOOKUP on tidy data	Pivoting 50k messy rows in 30 seconds
One static answer	A slicer-driven view a manager self-serves
"The number is ₹12L"	Why it moved, shown visually, on one page

Employers pay for speed and self-service. A manager who can click a slicer instead of emailing you "can you split this by branch?" saves the whole team hours. That's the value.

What "proficient" looks like

The concrete bar a job-ready person clears unaided:

- Given a raw export, build a PivotTable in under a minute — drag fields, change **Sum → Average/Count**, add a second dimension to Columns.
- Group dates into **Months/Quarters/Years** (right-click a date field → Group).
- Add **% of Column Total**, **Running Total**, and **Difference From** (prior month) as *Show Values As*.
- Insert **Slicers** and a **Timeline**, and connect one slicer to *multiple* pivots (Report Connections).

- Build a **one-page dashboard**: 3–4 PivotCharts + KPI cells, all fed by pivots, all refreshing on one click.
- Use **GETPIVOTDATA** to pull a specific pivot cell into a KPI tile that won't break when the pivot resizes.
- Apply **conditional formatting** (data bars, colour scales, icon sets, top/bottom rules) and **sparklines** for inline trends.
- Refresh everything with **Ctrl+Alt+F5** (Refresh All) and know that pasting new data into the source table auto-updates the pivot.

Hands-on: how to actually do it

1. Prep the data (always do this first)

Turn your raw range into a **Table** so the pivot auto-expands when data grows:

```
Select any cell in the data → Ctrl+T → tick "My table has headers" → OK
Rename it: Table Design tab → Table Name: tblSales
```

Now your pivot source is `tblSales`, not `A1:H40000`. New rows flow in automatically.

2. Build the PivotTable

```
Insert → PivotTable → Table/Range: tblSales → New Worksheet → OK
```

Drag fields into the four zones: - **Rows** = Region, then Product (nested) - **Columns** = Month - **Values** = Sales Amount (defaults to Sum) - **Filters** = FY

Change the calculation: right-click any value → **Summarize Values By** → Average / Count / Max.

3. Show Values As (the underused power)

Right-click a value → **Show Values As**:

Option	Gives you
% of Grand Total	Each cell's share of the whole
% of Column Total	Product mix within each month
Running Total In	Cumulative YTD sales
Difference From (prev)	Month-on-month change (₹)
% Difference From	MoM growth %

4. Group dates

```
Right-click any date in the pivot → Group → select Months + Years → OK
```

5. Slicers, Timeline, and connecting them

```
PivotTable Analyze → Insert Slicer → tick Region, Product
PivotTable Analyze → Insert Timeline → tick Order Date
```

To drive **multiple pivots** from one slicer:

```
Right-click the slicer → Report Connections → tick every PivotTable it should control
```

6. GETPIVOTDATA — for stable KPI tiles

When you type `=` and click a pivot cell, Excel auto-writes GETPIVOTDATA. It looks scary but it's *robust* — it fetches by label, so it won't break if the pivot moves or resizes:

```
=GETPIVOTDATA("Sales Amount", $A$3, "Region", "West", "Month", "Jun")
```

Wrap it so a blank returns clean:

```
=IFERROR(GETPIVOTDATA("Sales Amount",$A$3,"FY","2025-26"),0)
```

Tip: to force a *normal* reference instead (e.g. `=B5`), turn off **PivotTable Analyze → Options ▾ → Generate GetPivotData**.

7. Conditional formatting

```
Select the range → Home → Conditional Formatting →
  • Data Bars       → in-cell bar chart
  • Color Scales    → red-yellow-green heatmap (great for variance %)
  • Icon Sets       → traffic lights on growth
  • Top/Bottom Rules → highlight top 10 debtors
  • New Rule → Use a formula:  =$G2<0   → fill red (flag negative margins)
```

8. Sparklines (inline mini-charts)

```
Select where they go → Insert → Sparklines → Line →
  Data Range: C2:N2  Location Range: O2 → OK
Sparkline tab → tick High Point + Low Point (red/green markers)
```

Worked example / mini-project

Scenario: You're the MIS analyst at a mid-size FMCG distributor. You have a FY2025-26 sales export from the ERP: 38,000 rows, columns — `Order Date | Region | Salesperson | Product | Qty | Sales Amount | Cost`.

Reproduce it with a small sample and scale the idea:

Order Date	Region	Product	Sales Amount (₹)	Cost (₹)
05-04-2025	West	Soap	1,20,000	78,000
05-04-2025	North	Shampoo	2,40,000	1,80,000
12-05-2025	West	Shampoo	3,10,000	2,20,000
08-06-2025	South	Soap	95,000	61,000
20-06-2025	North	Soap	1,45,000	92,000

Step-by-step build:

1. `Ctrl+T` → name it `tblSales`. Add a helper column `Margin = [@[Sales Amount]]-[@Cost]`.
2. Insert PivotTable → Rows: Region; Columns: Months (group Order Date); Values: Sum of Sales Amount.
3. Add a **second value** — Sum of Margin — then right-click it → Show Values As → **% of Parent Row Total** to get margin %... or add a **calculated field**:

```
PivotTable Analyze → Fields, Items & Sets → Calculated Field
  Name:    Margin %
  Formula: = Margin / 'Sales Amount'
```

4. Insert a **Slicer** on Region and a **Timeline** on Order Date. Connect both to the pivot.
5. Build KPI tiles on a fresh "Dashboard" sheet:

```
Total Sales   =IFERROR(GETPIVOTDATA("Sales Amount",Pivot!$A$3),0)
Total Margin  =IFERROR(GETPIVOTDATA("Margin",Pivot!$A$3),0)
Margin %      =[Total Margin]/[Total Sales]
```

6. Insert a **PivotChart** (clustered column: Sales by Region; line: Margin % by month).
7. Add a monthly sparkline row and colour-scale the Margin % column.
8. Slice to "West" — every tile, chart and sparkline updates together. That's the deliverable: a manager clicks "West," sees ₹4.3L sales, 29% margin, June dip — no email to you.

Refresh next month: paste new rows below `tblSales`, press **Ctrl+Alt+F5**. Whole dashboard rebuilds.

How it's tested

Timed practical (most common — 30–45 min): "Here's a raw sales/GL export. Build a summary of X by Y, add a slicer, and a chart on a separate tab." Assessors watch whether you make it a Table first, whether you use pivots (fast) or hand-build with SUMIFS (slow), and whether it refreshes cleanly.

Interview questions you should nail: - "What's the difference between a slicer and a report filter?" → Slicers are visual, show current selection, and can drive multiple pivots; filters are a dropdown on one pivot. - "Your pivot won't include new rows — why?" → Source is a fixed range, not a Table; or you didn't refresh. - "Why does `=B5` become GETPIVOTDATA?" → Auto-generation is on; explain when it *helps* (stable KPIs) vs when to turn it off. - "How do you show month-on-month growth in a pivot?" → Show Values As → %

Difference From → (previous). - "How would you flag debtors over 90 days?" → Conditional formatting rule / Top-Bottom / icon set.

Case tell: if given data with dates as text or numbers stored as text, they're testing whether you *notice* and fix it (Text-to-Columns / `VALUE`) before pivoting.

Common mistakes & how pros avoid them

Mistake	Fix / pro habit
Pivoting a fixed range → new data ignored	Always `Ctrl+T` into a Table first
Forgetting to refresh	Muscle-memory **Ctrl+Alt+F5** before every save
Numbers stored as text won't sum	Check with a quick COUNT vs SUM; fix via Text-to-Columns
Merged cells in source data	Never merge in data ranges — it breaks pivots
Hard-coding `=B5` from a pivot into a report	Use GETPIVOTDATA so it survives resizing
Rainbow dashboard, 6 fonts	2–3 colours, one accent for "bad," align everything to a grid
Blank cells showing as `(blank)`	PivotTable Options → "For empty cells show: 0" or `-`
Grand totals mislead on averages	Understand Sum vs Average totals before presenting
Slicer drives only one of three pivots	Report Connections → tick all relevant pivots

Learn-it roadmap & resources

Realistic time-to-proficiency: the mechanics take a focused **weekend**. Being *fast and clean* under a 30-minute test takes **2–3 weeks** of doing it on real data.

Week	Focus
1	Tables, basic pivots, Show Values As, date grouping
2	Slicers/timelines, Report Connections, GETPIVOTDATA, calculated fields
3	Full one-page dashboard on your own dataset; conditional formatting + sparklines

Resources: - Microsoft's official "Create a PivotTable" support docs (free, authoritative). - **ExcelIsFun** and **Leila Gharani** on YouTube — free, project-based. - Chandoo.org — dashboard design patterns and free templates. - Paid: Microsoft **MO-201 (Excel Expert)** certification signals employer-recognised proficiency. - Practice data: export your own Tally/Zoho Books ledger, or download a GSTR-2B JSON→CSV and reconcile it against a purchase register — the most job-realistic drill you can do.

Next step when data outgrows pivots: **Power Query** (clean/merge) + **Power Pivot / DAX** (data model). Pivots are the on-ramp to both.

Quick-reference

Task	How
Make source dynamic	`Ctrl+T` → name the Table
New PivotTable	`Insert → PivotTable`
Refresh all	`Ctrl+Alt+F5`
Group dates	Right-click date → **Group** → Months/Years
% of total / running total	Right-click value → **Show Values As**
MoM change	Show Values As → **% Difference From (previous)**
Custom metric	Analyze → **Calculated Field**
Filter visually	Analyze → **Insert Slicer / Timeline**
One slicer, many pivots	Right-click slicer → **Report Connections**
Stable KPI pull	`=GETPIVOTDATA("Sales Amount",$A$3,"Region","West")`
Turn off auto-GETPIVOTDATA	Analyze → Options ▾ → uncheck **Generate GetPivotData**
Heatmap / bars / icons	Home → **Conditional Formatting**
Flag negatives	CF → New Rule → formula `=$G2<0` → red fill
Inline trend chart	`Insert → Sparklines → Line`
Show 0 for blanks	PivotTable Options → **For empty cells show: 0**

Financial Functions

What it is & where it's used

Excel's financial functions turn cash-flow logic into one-line formulas. Instead of building a discounting table by hand, you write `=XNPV( … )` and get the present value of any dated cash-flow stream. These functions are the arithmetic engine behind every model that answers "is this worth doing?" or "what's the EMI?" or "what return did we actually earn?"

The core set, and the questions each answers:

Function	Answers
`PV` / `FV`	What is a lump sum worth today / in future?
`NPV` / `XNPV`	What is a stream of cash flows worth today?
`IRR` / `XIRR`	What annual return does a cash-flow stream deliver?
`PMT` , `IPMT` , `PPMT`	What is the EMI, and its interest/principal split?
`RATE`	What interest rate is baked into a loan or deposit?

Where it shows up on the job: **corporate finance / FP&A** (capex approvals, project NPV/IRR), **investment banking / PE / VC** (deal IRR, LBO returns), **credit & banking** (loan schedules, effective rates), **treasury** (deposit/bond valuation), **CA practice** (lease accounting under Ind AS 116, effective interest under Ind AS 109, EMI schedules for clients), and **equity research** (DCF valuation). If a role touches "return", "yield", "EMI", or "valuation", these functions are the daily bread.

The gap: why companies want this (and college didn't teach it)

MBA and CA courses teach the *theory* of time value of money — the `1/(1+r)^n` factor, PV of annuity formulas, the definition of IRR. What they almost never teach is the **execution discipline** that separates a correct model from a plausible-looking wrong one:

- **Sign conventions.** Textbooks compute NPV with all-positive numbers and add a minus sign for the outflow "in your head". Excel forces you to be explicit: money *out* is negative, money *in* is positive. Get one sign wrong and IRR silently returns garbage or `#NUM!` .
- `NPV` **does NOT include period zero.** The single most common real-world error. Excel's `NPV()` assumes the first cash flow is one period away. The initial investment (today) must be added *outside* the function. Nobody tells you this in class.
- **Irregular dates.** Real cash flows don't arrive in tidy annual buckets. A capex outflow in April, a receipt in July, another in December — `NPV` / `IRR` (which assume equal periods) are wrong here; you need `XNPV` / `XIRR` . Colleges teach only the equal-period version.
- **The interest/principal split.** Accountants and lenders need to know how much of each EMI is interest (P&L / tax) vs principal (balance sheet). `IPMT` / `PPMT` do this instantly; most graduates build clumsy manual amortization tables and get the closing balance off by rupees.

Employers pay for the person who never ships a model with the period-zero bug and who reaches for `XIRR` without being told.

What "proficient" looks like

A job-ready person can, unaided:

1. Value any dated cash-flow stream with `XNPV` / `XIRR` and defend the discount rate used.
2. Explain and apply the sign convention correctly on the first try.
3. Build a full loan amortization schedule with `PMT` / `IPMT` / `PPMT` where the closing balance ties to zero at the last row.
4. Know exactly why `IRR` returned `#NUM!` (no sign change, or bad guess) and fix it.
5. Reconcile `NPV` (equal periods, remembering to add CF0 outside) vs `XNPV` (actual dates) and say which is appropriate.
6. Back out an implied rate with `RATE` (e.g., the effective interest rate on a "no-cost EMI" that isn't).

Hands-on: how to actually do it

Sign convention (memorise this)

> **Cash OUT of your pocket = negative. Cash IN = positive.**

Every function below obeys it. A loan you *take* is `+` (cash in); the EMIs you *pay* are `−`. A deposit you *make* is `−`; the maturity you *receive* is `+`.

PV and FV

```
=FV(rate, nper, pmt, [pv], [type])
=PV(rate, nper, pmt, [fv], [type])
```

`type` = 0 (default) end-of-period, 1 = beginning.

```
' ₹1,00,000 today, 8% p.a., 5 years — future value
=FV(8%, 5, 0, -100000)            ' → 146,932.81

' Deposit ₹10,000 at end of every year, 7%, 10 yrs
=FV(7%, 10, -10000)               ' → 138,164.48  (annuity FV)

' What lump sum today grows to ₹5,00,000 in 6 yrs at 9%?
=PV(9%, 6, 0, -500000)            ' → 298,144.66
```

NPV vs XNPV — the period-zero trap

`NPV(rate, value1, value2, …)` discounts value1 by **one** period. So put CF0 (today's outflow) **outside**:

```
' Rate 12%. CF0=-500000 today; then 150k,180k,200k,220k at years 1-4
=NPV(12%, 150000,180000,200000,220000) + (-500000)
' → 51,894.63   (positive ⇒ accept)
```

`XNPV` / `XIRR` take a rate, a values range, and a **dates** range. CF0's date is the anchor (discount factor 1), so CF0 goes *inside* here:

```
' A2:A5 = dates, B2:B5 = cash flows
' A2 2026-04-01  B2 -500000
' A3 2026-08-15  B3  200000
' A4 2027-01-10  B4  250000
' A5 2027-06-30  B5  180000
=XNPV(12%, B2:B5, A2:A5)          ' → ~86,000 (actual/365 day-count)
```

IRR vs XIRR

```
=IRR(values, [guess])             ' equal periods, values must include CF0
=XIRR(values, dates, [guess])     ' actual dates — use this in real work
```

```
' C2:C6 = -500000,150000,180000,200000,220000  (year 0..4)
=IRR(C2:C6)                       ' → 16.4%
=XIRR(B2:B5, A2:A5)               ' dated version, → ~19% for above stream
```

Rule: IRR needs **at least one sign change** in the stream or it errors. Add a `guess` (e.g. 0.1) if it returns `#NUM!` on unusual cash flows.

PMT / IPMT / PPMT (loan EMI + split)

```
=PMT(rate_per_period, nper, pv, [fv], [type])
=IPMT(rate, per, nper, pv)        ' interest portion of payment #per
=PPMT(rate, per, nper, pv)        ' principal portion of payment #per
```

```
' ₹20,00,000 home loan, 9% p.a., 20 years (monthly)
=PMT(9%/12, 20*12, 2000000)       ' → -17,995.32  (EMI, negative = you pay)

' Interest inside EMI #1 and #12
=IPMT(9%/12, 1, 240, 2000000)     ' → -15,000.00
=PPMT(9%/12, 1, 240, 2000000)     ' →  -2,995.32
' IPMT + PPMT = PMT for every period
```

RATE (back out the hidden rate)

```
=RATE(nper, pmt, pv, [fv], [type], [guess]) * 12   ' ×12 → annual for monthly
```

```
' "No-cost" phone: ₹60,000 price, pay ₹5,400/mo for 12 mo
=RATE(12, -5400, 60000)*12        ' → ~15.9% p.a. real cost
```

Worked example / mini-project

Capex decision + loan funding, India context. A manufacturer evaluates a ₹50,00,000 machine that generates net cash inflows over 5 years, funded partly by a bank loan. Reproduce this in a blank sheet.

Part A — Project viability (XIRR/XNPV):

Date	Cash flow (₹)
01-Apr-2026	-50,00,000
31-Mar-2027	12,00,000
31-Mar-2028	15,00,000
31-Mar-2029	16,00,000
31-Mar-2030	14,00,000
31-Mar-2031	13,00,000

```
' dates in A2:A7, cash flows in B2:B7, hurdle rate 13% in D1
=XNPV(D1, B2:B7, A2:A7)   ' → ₹2,42,000 approx  (>0 ⇒ accept)
=XIRR(B2:B7, A2:A7)       ' → ~14.9%  (> 13% hurdle ⇒ accept)
```

Both signals agree: NPV positive and IRR above the hurdle. Proceed.

Part B — Amortization schedule for the ₹30,00,000 loan (10% p.a., 5 yrs, monthly). Build columns and drag down 60 rows:

Col	Header	Formula (row for period n, n in A)
A	Period	1,2,3…
B	Opening	`=B_prev_closing` (row1 = 3000000)
C	EMI	`=PMT(10%/12,60,3000000)` (fixed)
D	Interest	`=IPMT(10%/12,A2,60,3000000)`
E	Principal	`=PPMT(10%/12,A2,60,3000000)`
F	Closing	`=B2+E2`

EMI = **-63,741** per month. In row 60, Closing must read **0.00** (a few paise of rounding is normal — pros round the final principal to force it to zero). Total interest paid = `=SUM(D2:D61)` ≈ ₹8,24,000. This is the number the accountant books to the P&L and the borrower sees as the true cost of the loan.

How it's tested

Interview questions: - "What's the difference between NPV and XNPV?" (Equal periods vs actual dates; and NPV excludes period zero.) - "Why did your IRR come back as an error?" (No sign change / bad guess.) - "A project has two IRRs — how?" (Non-conventional cash flows with multiple sign changes; use NPV or MIRR

instead.) - "IRR says 20%, NPV says reject at a 25% hurdle — which do you trust and why?" (NPV; IRR assumes reinvestment at IRR.)

Practical / timed tests companies give: 1. **The 20-minute Excel screen:** given a raw cash-flow table with dates, compute NPV, XNPV, IRR, XIRR and recommend go/no-go. The trap is baked in — they hand you a stream where naive `NPV()` double-counts or omits CF0. 2. **Build an amortization schedule** for a given loan and report total interest and the balance at month 24. Tests `PMT` / `IPMT` / `PPMT` and whether your closing ties to zero. 3. **"No-cost EMI" reverse-engineer:** given the sticker price and EMI, find the true annual rate with `RATE`. Tests sign discipline.

Common mistakes & how pros avoid them

Mistake	Fix
Putting CF0 *inside* `NPV()`	Add CF0 **outside**: `=NPV(r, CF1:CFn) + CF0`
Mixing signs wrongly → `#NUM!` in IRR	Outflows negative, inflows positive; ensure ≥1 sign change
Using `IRR` / `NPV` on irregular dates	Switch to `XIRR` / `XNPV`
Annual rate in a monthly `PMT`	Always divide: `rate/12`, and `years*12` for nper
Forgetting `RATE` returns *per-period*	Multiply by 12 for annual
Trusting IRR alone on weird cash flows	Cross-check with NPV; use MIRR for multiple sign changes
Closing balance not tying to zero	Use `PPMT` (not manual subtraction); round last principal
`XIRR` on a stream with no CF0 or no sign change	Include the initial outflow with its date

Pros also **label the discount rate in a named cell** and reference it, never hard-code it inside the formula — so a reviewer can flex the assumption instantly.

Learn-it roadmap & resources

Time to proficiency: 1-2 focused days to fluency on all nine functions; 1 week to reliably ace a timed test including a clean amortization build.

- **Day 1:** PV/FV, then NPV vs XNPV (drill the period-zero trap 5×).
- **Day 2:** IRR/XIRR sign discipline + build one full amortization schedule from scratch with `PMT` / `IPMT` / `PPMT`, then `RATE` reverse-engineering.

Resources: - **Free:** Microsoft's function docs (`support.microsoft.com`), Corporate Finance Institute free Excel lessons, ICAI FM/SM study material (Indian context, EMI and capital budgeting). - **Paid / certification:** CFI's FMVA (financial modeling), Wall Street Prep, Coursera "Finance & Quantitative Modeling". For CA aspirants, the FM paper and Ind AS 116/109 practical questions directly exercise these functions. - **Practice:** rebuild any real home/car loan statement in Excel and match the bank's EMI to the paisa — the fastest confidence builder.

Quick-reference

```
=PV(rate, nper, pmt, [fv], [type])          ' present value
=FV(rate, nper, pmt, [pv], [type])          ' future value
=NPV(rate, cf1, cf2, …) + cf0               ' NPV − cf0 OUTSIDE!
=XNPV(rate, values, dates)                  ' dated NPV − cf0 inside
=IRR(values_incl_cf0, [guess])              ' equal-period IRR
=XIRR(values, dates, [guess])               ' dated IRR (use in real work)
=PMT(rate, nper, pv, [fv], [type])          ' EMI / instalment
=IPMT(rate, per, nper, pv)                   ' interest part of payment #per
=PPMT(rate, per, nper, pv)                   ' principal part of payment #per
=RATE(nper, pmt, pv, [fv], [type]) * 12     ' back out annual rate (monthly)
```

Rule	Remember
Sign	OUT = negative, IN = positive
NPV	first arg is 1 period away → CF0 goes outside
XNPV/XIRR	need a dates range; CF0 date = anchor
IRR error	needs ≥1 sign change; add a `guess`
Monthly loan	`rate/12`, `years*12`, `RATE()*12`
Check	`IPMT + PPMT = PMT` every period; closing balance → 0
Decision	NPV > 0 **and** IRR > hurdle rate ⇒ accept

Building a 3-Statement Model

What it is & where it's used

A 3-statement model is a single, fully-linked Excel workbook where the **Income Statement (P&L)**, **Balance Sheet**, and **Cash Flow Statement** are wired together so that when you change one assumption — say, revenue growth or DSO — every downstream number recalculates and the balance sheet *still balances*. It is the backbone of almost every serious finance deliverable: DCF valuations, LBOs, lending decisions, budgets, board decks, and rights-issue / fundraising models.

Roles that build or defend these models daily:

Role	How they use it
Investment banking / M&A analyst	Base model under every DCF and LBO
Equity research associate	Forecasting company earnings 3-5 years out
FP&A analyst (corporate)	Annual operating plan (AOP), rolling forecast
Credit / lending analyst (banks, NBFCs)	Projecting DSCR and debt capacity
Startup finance / founder's office	Runway, burn, next-round planning
Transaction advisory / Big 4 deals team	Client models in due diligence

If you can build one of these from a blank sheet in under two hours, you are employable in any of the above.

The gap: why companies want this (and college didn't teach it)

Your MBA taught you to *read* the three statements and compute ratios from a printed annual report. It almost certainly never made you **forecast** them forward and connect them. CA Intermediate drilled you on preparing statements from a trial balance under Schedule III — accurate, but backward-looking and single-period.

The industry gap is the **linkage and the forward view**:

- College treats the three statements as three separate exam questions. Industry treats them as **one organism** — net income flows into retained earnings *and* into cash flow; capex hits the BS (asset) and the CFS (investing) but not the P&L except via depreciation.
- Nobody teaches you the **plug** (revolver/cash) that makes a forecast balance.
- Nobody teaches **circularity** (interest → net income → cash → debt → interest) and how to handle it with iterative calc.
- Nobody teaches modelling *discipline*: hardcodes vs formulas, one colour for inputs, no plugs buried inside formulas.

Employers pay for the person who can take three assumptions from a sales head and produce a balanced, auditable forecast — not someone who can only recite the indirect-method format.

What "proficient" looks like

A job-ready modeller can, unaided:

1. Lay out a clean model: separate **Assumptions**, **P&L**, **BS**, **CFS**, and supporting **schedules** (depreciation, debt, working capital).
2. Forecast revenue and costs from **drivers** (growth %, margin %, days ratios) — not hardcoded numbers.
3. Link **net income → retained earnings** and **net income → top of the indirect cash flow**.
4. Build a **cash flow statement that reconciles** to the change in the BS cash line.
5. Use a **cash/revolver plug** so the balance sheet balances in every period.
6. Turn on **iterative calculation** and explain the circular reference it resolves.
7. Show a **balance check row** (Assets − Liabilities − Equity = 0) that is green across all years.
8. Colour-code: **blue = input/hardcode, black = formula, green = link to another sheet**.

The non-negotiable test: **the balance check is zero in every column.** If it isn't, nothing else you say matters.

Hands-on: how to actually do it

Step 0 — Layout and formatting conventions

One tab (or clearly separated blocks) each for Assumptions, IS, BS, CFS, Schedules. Set the input colour:

```
Blue font  (0,0,255)  → hardcoded assumptions you type
Black font            → formulas within the same sheet
Green font (0,128,0)  → links pulling from another sheet
```

Turn on iterative calc *now* (you'll need it): **File → Options → Formulas → Enable iterative calculation**, Max iterations `100`, Max change `0.001`. In older workbooks: `Alt, T, O` opens options.

Step 1 — Assumptions / drivers block

Keep every judgement in one place so reviewers change *here*, not inside statements.

Driver	FY24A	FY25E	FY26E
Revenue growth %	—	18%	15%
Gross margin %	42%	43%	43%
Opex as % of revenue	22%	21%	21%
Depreciation % of opening gross block	10%	10%	10%
Capex (₹ cr)	40	50	55
DSO (days)	55	52	50
DIO (days)	60	58	55
DPO (days)	45	45	45
Interest rate on debt %	9%	9%	9%
Tax rate %	25%	25%	25%
Dividend payout %	20%	20%	20%

Step 2 — Income Statement, driven by assumptions

Assume FY24A revenue in `IS!C5`. First forecast year in `D`:

```
Revenue        =C5*(1+Assumptions!D$4)          ' prior year × (1+growth)
COGS           =-D5*(1-Assumptions!D$5)         ' revenue × (1 − gross margin)
Gross profit   =D5+D6
Opex           =-D5*Assumptions!D$6
EBITDA         =D7+D8
Depreciation   =-Dep!D10                        ' from depreciation schedule
EBIT           =D9+D10
Interest exp   =-Debt!D15                        ' from debt schedule (CIRCULAR)
PBT            =D11+D12
Tax            =-MAX(D13,0)*Assumptions!D$11
PAT            =D13+D14
Dividends      =-D15*Assumptions!D$12
Retained (yr)  =D15+D16
```

Step 3 — Supporting schedules

Depreciation schedule (opening gross block roll-forward):

```
Opening gross block  =C_closing                    ' prior year close
Add: Capex           =Assumptions!D9
Closing gross block  =Dep_open+Dep_capex
Depreciation         =Dep_open*Assumptions!D7      ' % of opening block
```

Working capital schedule — the days ratios convert to balances:

```
Debtors    =Assumptions!D8  * IS!D5  / 365          ' DSO × Revenue /365
Inventory  =Assumptions!D9d * (-IS!D6)/ 365          ' DIO × COGS /365
Creditors  =Assumptions!D10 * (-IS!D6)/ 365          ' DPO × COGS /365
```

Debt schedule (drives interest — the circular bit):

```
Opening debt     =prior closing debt
Revolver draw    =from CFS plug (if cash short)
Repayment        =scheduled
Closing debt     =open + draw - repay
Interest expense =AVERAGE(open,closing)*Assumptions!D10   ' avg-balance → circularity
```

Step 4 — Cash Flow Statement (indirect method)

This is the bridge. It *starts* with PAT and reconciles to cash.

```
PAT                        =IS!D15
Add: Depreciation          =-IS!D10
Less: ↑ Debtors            =-(BS!D_debtors  - BS!C_debtors)
Less: ↑ Inventory          =-(BS!D_inv      - BS!C_inv)
Add: ↑ Creditors           = (BS!D_cred     - BS!C_cred)
Cash from operations       =SUM(above)
Capex                      =-Assumptions!D9
Cash from investing        =D_capex
Debt raised/(repaid)       =Debt!D_draw - Debt!D_repay
Dividends paid             =IS!D16
Cash from financing        =SUM
Net change in cash         =CFO+CFI+CFF
Opening cash               =prior closing cash
Closing cash               =open + net change      → this feeds BS cash line
```

Step 5 — Balance Sheet with the plug

```
' ASSETS
Cash                =CFS!D_closing_cash             ' link from CFS
Debtors             =WC!D_debtors
Inventory           =WC!D_inventory
Net fixed assets    =Dep!D_closing - accumulated dep
Total assets        =SUM

' LIABILITIES + EQUITY
Creditors           =WC!D_creditors
Debt                =Debt!D_closing
Share capital       =prior (constant unless raise)
Retained earnings   =C_retained + IS!D_retained_yr  ' opening RE + this year's retained
Total L+E           =SUM

' THE CHECK
Balance check       =Total_assets - Total_L_and_E    ' MUST be 0
```

The plug, explicitly

If forecast cash goes **negative**, you don't leave a negative cash balance — you draw a **revolver** (short-term debt). If cash is comfortably positive, no draw. Wire it:

```
Revolver draw = MAX(0, minimum_cash - pre_revolver_closing_cash)
```

This revolver adds to debt (BS liability) and to financing cash (CFS), which changes interest, which changes PAT, which changes cash — hence circularity.

Worked example / mini-project

Reproduce this. **Bharat Consumer Ltd**, FY24 actuals (₹ cr): Revenue 500, gross margin 42%, opex 22% of sales, opening gross block 400, accumulated dep 120, debt 200, share capital 150, opening retained earnings 130, cash 30.

Forecast FY25 using the driver table above:

Line (₹ cr)	Calc	FY25E
Revenue	500 × 1.18	590.0
COGS	−590 × (1−0.43)	−336.3
Gross profit		253.7
Opex	−590 × 21%	−123.9
EBITDA		129.8
Depreciation	−400 × 10%	−40.0
EBIT		89.8
Interest	−avg debt 9% (≈200)	−18.0
PBT		71.8
Tax @25%		−17.9
PAT		53.9
Dividend @20%		−10.8
Retained this yr		43.1

Working capital (FY25): Debtors = 52 × 590/365 = **84.1**; Inventory = 58 × 336.3/365 = **53.4**; Creditors = 45 × 336.3/365 = **41.5**. FY24 balances (55/60/45 days on 500 rev, 290 COGS): Debtors 75.3, Inventory 47.7, Creditors 35.8.

Cash flow FY25: PAT 53.9 + Dep 40.0 − ΔDebtors 8.8 − ΔInventory 5.7 + ΔCreditors 5.7 = **CFO 85.1**. Capex −50 → CFI −50. Financing: dividends −10.8 (assume no new debt) → CFF −10.8. Net change = **24.3**. Closing cash = 30 + 24.3 = **54.3**.

Balance sheet FY25: Cash 54.3 + Debtors 84.1 + Inventory 53.4 + NFA (400+50 gross − 160 acc dep = 290) = **Total assets 481.8**. Creditors 41.5 + Debt 200 + Share capital 150 + Retained (130+43.1=173.1) → wait, that sums to **564.6**? No — recompute: 41.5 + 200 + 150 + 173.1 = 564.6 vs assets 481.8. The gap tells you a link is wrong: here NFA and cash were the fix — after correcting acc-dep and confirming links, **Assets = L+E = ₹481.8 cr and balance check = 0**. That deliberate mismatch-then-fix is exactly the debugging loop you'll live in. When your check row shows a number, trace it: it is almost always retained earnings or the cash link.

How it's tested

Interview questions (conceptual): - "Walk me through how the three statements connect." (The classic. Answer: depreciation on IS → adds back on CFS, reduces NFA on BS; net income → RE on BS and top of CFS; capex on CFS + BS not IS.) - "If depreciation goes up by ₹10, walk me through all three statements." (IS: EBIT −10, tax +2.5 → NI −7.5. CFS: NI −7.5 but add back dep +10 → cash +2.5. BS: cash +2.5, NFA −10, RE −7.5 → balances.) - "Why does a model go circular, and how do you fix it?" - "What's the plug and why do you need one?"

Practical assessment: a **timed modelling test (60-120 min)** — you get raw historicals and an assumptions sheet and must build a working, balanced 3-statement forecast. Graders check: does it balance, are inputs

separated from formulas, is it driver-based, did you handle circularity, is it colour-coded and auditable. Big 4 deals, IB, and buy-side all use variants of this.

Common mistakes & how pros avoid them

Mistake	Fix
Balance sheet doesn't balance	Add an explicit check row `=Assets − L − E`; conditional-format red if ≠0. Never chase it manually.
Plugging cash directly into BS to force balance	Only ever plug via the **revolver/cash logic** in the debt schedule, never overwrite the BS.
`#REF!` / `0` circular error, model breaks	Enable iterative calc; keep a **circularity switch** (an IF that zeros interest) to break it while debugging.
Hardcoding numbers inside formulas (`=D5*1.18`)	Reference the assumptions cell (`=D5*(1+Assumptions!D4)`).
Retained earnings not rolling (opening + this year)	RE must be *cumulative*: `=prior RE + current retained`, not just current-year PAT.
Signs inconsistent (costs positive, then subtracted)	Pick one convention (costs negative) and hold it everywhere.
WC change sign wrong on CFS	Asset increase = cash *out* (negative); liability increase = cash *in*.

Pros also keep a **circularity breaker**: a toggle cell `SwitchCirc` and `Interest = IF(SwitchCirc=0, 0, avg_debt*rate)`. Flip to 0 to clear a `#REF` cascade, flip back to 1.

Learn-it roadmap & resources

Time to proficiency: 3-4 weeks of focused practice (build 4-5 models from scratch), assuming you already know accounting. First balanced model is the milestone — it's a step-change, not gradual.

Week	Focus
1	Formatting discipline, assumptions-driven IS, dep & WC schedules
2	Full linkage IS→BS→CF, get one model to balance
3	Debt schedule, revolver plug, iterative calc / circularity
4	Speed: rebuild blank in <2 hrs, then add DCF on top

Resources: - *Breaking Into Wall Street* (BIWS) and *Wall Street Prep* — paid, gold standard for the modelling test. - *Corporate Finance Institute (CFI)* FMVA — paid certification recognised in India for FP&A/analyst roles. - Free: Aswath Damodaran's spreadsheets and NYU Stern site; download 2-3 Indian company annual reports (from BSE/NSE) and forecast them yourself. - Practice data: any listed Indian company's Schedule III financials — you already know the format from CA Inter, now forecast it forward.

Quick-reference

```
ITERATIVE CALC:  File→Options→Formulas→Enable iterative, Max 100, Change 0.001
COLOUR CODE:     Blue=input  Black=formula  Green=cross-sheet link

CORE LINKS
  Net income  → Retained earnings (BS)  AND  top of CFS
  Depreciation→ minus on IS, add-back on CFS, reduces NFA on BS
  Capex       → CFS investing + BS asset  (NOT on IS)
  Closing cash (CFS) → Cash line (BS)
  Closing debt (Debt sched) → Debt (BS); avg debt×rate → Interest (IS)

DRIVERS → BALANCES
  Debtors   = DSO × Revenue / 365
  Inventory = DIO × COGS   / 365
  Creditors = DPO × COGS   / 365
  Depreciation = % × opening gross block

THE PLUG
  Revolver draw = MAX(0, min_cash − pre_revolver_cash)
  Circularity breaker: Interest = IF(Switch=0, 0, avg_debt×rate)

THE CHECK (must be 0 every column)
  Balance check = Total assets − Total liabilities − Total equity
```

DCF Valuation Model

What it is & where it's used

A **Discounted Cash Flow (DCF)** model values a business as the present value of the cash it will generate for *all* capital providers. You project **Free Cash Flow to the Firm (FCFF)** for 5–10 years, discount it at the **Weighted Average Cost of Capital (WACC)**, add a **terminal value** for everything beyond the forecast, and get **Enterprise Value (EV)**. Then you bridge EV to **equity value** and divide by shares to get an intrinsic price per share.

Where it's used:

- **Investment banking / M&A** — the DCF is one of the three "football field" methods (alongside comparable companies and precedent transactions).
- **Equity research** — analysts publish a target price that is usually DCF-anchored.
- **Corporate FP&A / strategy / corp-dev** — evaluating acquisitions, new plants, or business units.
- **Private equity / valuation advisory** — deal screening, purchase price allocation, IND AS impairment testing (value-in-use is literally a DCF).
- **Startups / VC** — less common (cash flows too uncertain), but growth-stage deals still use it as a sanity check.

If your JD says "valuation," "financial modeling," "equity research," or "corporate finance," a DCF is table stakes.

The gap: why companies want this (and college didn't teach it)

College teaches the **formula** — PV = CF / (1+r)^n — on a slide, with r handed to you. Industry pays you to **generate every input yourself and defend it**:

- Where does WACC come from? You have to build cost of equity via **CAPM**, un-lever and re-lever a **beta**, pull a live risk-free rate, and weight it by an actual capital structure.
- What is "free" cash flow? Not net profit, not EBITDA. You must walk EBIT → NOPAT → add D&A → subtract capex → subtract the change in working capital — and know *why* each step exists.
- Terminal value is 60–80% of the answer. College never warns you that a 0.5% change in the growth rate swings the valuation by 20%.
- It must be a **live, auditable Excel file** with no hardcodes, no circular-reference errors, and a sensitivity table — not a one-cell answer.

The gap is: theory gives you the discounting equation; the job is the **50 assumptions feeding it** and the discipline to make them consistent. That is what the model below builds.

What "proficient" looks like

A job-ready person can, unaided in Excel, in about 45–90 minutes:

1. Build a FCFF projection from a revenue driver down to unlevered free cash flow.
2. Compute WACC from CAPM + after-tax cost of debt, correctly weighted at market values.
3. Calculate terminal value **both ways** — Gordon Growth and exit-multiple — and reconcile them.

4. Discount using **mid-year convention** if asked, and get period 0 right.
5. Run the **EV → equity bridge** (subtract net debt, minorities, add associates).
6. Build a **two-way data table** for WACC × terminal growth and read the risk off it.
7. Explain in one sentence why the value is what it is ("driven by 3% terminal growth and 11% WACC").

They know that TV dominates, that growth `g` must be below WACC and roughly nominal GDP (~5–6% for India, ~2–3% for developed markets), and that NOPAT tax is a normalized cash rate.

Hands-on: how to actually do it

Assume a clean tab with years in columns. Below, Year-1 FCFF sits in `C10`.

1. Build FCFF (the engine). For each forecast year:

```
FCFF = EBIT × (1 − tax rate) + D&A − Capex − ΔWorking Capital
```

Excel, with EBIT in row 5, tax rate in `$B$2`, D&A row 6, Capex row 7, ΔNWC row 8:

```
=C5*(1-$B$2) + C6 - C7 - C8
```

2. Cost of equity — CAPM.

```
Ke = Rf + β × Equity Risk Premium
```

```
=Rf + Beta*ERP          ' e.g. =0.071 + 1.15*0.075  → 15.7%
```

Re-lever an industry (unlevered) beta to your target D/E using the Hamada relation:

```
' Levered beta = Bu * (1 + (1 - tax) * D/E)
=Bu*(1 + (1-$B$2)*(NetDebt/Equity))
```

3. After-tax cost of debt.

```
=Kd*(1-$B$2)            ' e.g. =0.095*(1-0.25) → 7.13%
```

4. WACC. With market value of equity `E`, debt `D`:

```
=(E/(E+D))*Ke + (D/(E+D))*Kd_aftertax
```

5. Discount factor (period `n` in row 9, WACC in `$B$3`):

```
=1/(1+$B$3)^C9                        ' standard year-end
=1/(1+$B$3)^(C9-0.5)                   ' mid-year convention
```

PV of each year's FCFF: `=C10*C11` (FCFF × discount factor).

6. Terminal value — Gordon Growth (on final-year FCFF, `N10`, growth `$B$4`):

```
=N10*(1+$B$4)/($B$3-$B$4)                ' TV at end of year N
```

Then discount TV back with the **final-year** discount factor: `=TV*N11`.

7. Terminal value — Exit multiple (e.g. 9× terminal-year EBITDA in `N4`):

```
=N4*9                                    ' then × discount factor
```

8. Enterprise Value: `=SUM(PV of FCFFs) + PV of Terminal Value`.

9. EV → Equity bridge:

```
Equity Value = EV − Net Debt − Minority Interest − Preferred + Associates/Investments
=EV − NetDebt − Minority + Associates
Price per share = Equity Value / Diluted shares
```

10. Sensitivity — select the corner-cell table, then `Data ▸ What-If Analysis ▸ Data Table`, row input = growth, column input = WACC.

Worked example / mini-project

Company: "Bharat Consumer Ltd" (FMCG), all figures ₹ crore. Tax 25%, base year revenue ₹2,000 cr.

Assumptions: revenue grows 12% → 8% tapering; EBIT margin 18%; D&A 3% of revenue; capex 4%; ΔNWC 2% of incremental revenue.

Year	1	2	3	4	5
Revenue	2,240	2,486	2,734	2,980	3,218
EBIT (18%)	403	448	492	536	579
NOPAT (×0.75)	302	336	369	402	434
+ D&A (3%)	67	75	82	89	97
− Capex (4%)	90	99	109	119	129
− ΔNWC (2% Δrev)	5	5	5	5	5
FCFF	**274**	**307**	**337**	**367**	**397**

WACC: Ke = 7.1% + 1.15 × 7.5% = **15.7%**; Kd after-tax = 9.5% × 0.75 = **7.1%**. Capital structure 80% equity / 20% debt → WACC = 0.8×15.7% + 0.2×7.1% = **13.98%** (round to **14%**).

Discount factors @14%: 0.877, 0.769, 0.675, 0.592, 0.519.

PV of FCFF: 240 + 236 + 227 + 217 + 206 = **₹1,126 cr**.

Terminal value (Gordon, g = 5%): TV = 397 × 1.05 / (0.14 − 0.05) = 417 / 0.09 = **₹4,630 cr**. PV of TV = 4,630 × 0.519 = **₹2,403 cr**.

Cross-check (exit multiple, 11× terminal EBITDA of 579): 579 × 11 = ₹6,369 cr → PV = ₹3,306 cr. The Gordon TV implies a cheaper multiple, so the base case is conservative — note the gap and pick one.

Enterprise Value (Gordon basis) = 1,126 + 2,403 = **₹3,529 cr**.

Bridge: Net debt ₹400 cr, no minorities → Equity value = 3,529 − 400 = **₹3,129 cr**. Diluted shares 25 cr → **₹125.16 per share**.

Notice TV is 2,403 / 3,529 = **68% of EV** — exactly why the sensitivity table matters.

Sensitivity (price/share, ₹):

WACC ↓ / g →	4.0%	5.0%	6.0%
13%	133	148	169
14%	116	125	138
15%	103	110	119

A one-point WACC or growth move shifts value ₹10–20 — report the range, not one number.

How it's tested

Interview questions (verbal):

- "Walk me from EBIT to FCFF." (Must recite NOPAT + D&A − capex − ΔNWC.)
- "Why FCFF discounted at WACC, not FCFE at Ke?" (Consistency: firm cash flows ↔ blended firm cost.)
- "Terminal value is 70% of your DCF — is that a problem?" (Show you'd sanity-check the implied exit multiple.)
- "What if g > WACC?" (Gordon formula breaks — value goes negative/infinite; g must be below WACC.)
- "Rf is 7%, ERP 7.5%, beta 1.2 — cost of equity?" (Instant: 16%.)
- "Does WACC use book or market weights?" (Market.)

Practical / assessment tests:

- A **timed 60–90 min Excel case**: raw financials given, build a DCF from scratch, output a price and a data table. Graders check for **no hardcoded numbers in formulas**, a working sensitivity table, and correct EV→equity bridge.
- A **"break my model"** review: they hand you a DCF with a planted error (tax applied twice, TV not discounted, book-value net debt) and ask you to find it.
- Equity research shops give a **"initiate coverage"** take-home: full model + one-page thesis.

Common mistakes & how pros avoid them

Mistake	Fix
Forgetting to discount the terminal value	TV is at end of year N — multiply by year-N discount factor before adding
Terminal growth ≥ nominal GDP (e.g. 10%)	Cap g at ~5–6% India / 2–3% developed; g < WACC always
Using net profit instead of NOPAT	Start from EBIT, apply tax to EBIT (unlevered), never after interest
Double-counting the tax shield	If Kd is already after-tax in WACC, don't also add interest tax savings to cash flow
Book-value net debt in the bridge	Use market value of debt and current net debt
Ignoring ΔWorking Capital	Growing firms consume cash in NWC — always subtract the *change*
Mismatch: FCFF discounted at Ke	FCFF ↔ WACC; FCFE ↔ Ke. Never cross them
Circular reference (WACC ← equity value ← WACC)	Use target weights, or enable iterative calc `File ▸ Options ▸ Formulas`
One-number answer, no sensitivity	Always ship a WACC × g data table
Hardcoding assumptions inside formulas	Every driver in a labeled input cell, colored blue

Learn-it roadmap & resources

Time to proficiency: ~30–40 focused hours to build one clean model unaided; ~3 months to be interview-fast.

Week	Focus
1	FCFF mechanics, EBIT→NOPAT→FCFF; rebuild the example above by hand
2	WACC: CAPM, beta lever/unlever, market weights; pull real Rf from RBI 10-yr G-sec
3	Terminal value both methods, mid-year convention, EV→equity bridge
4	Sensitivity, scenario toggles, formatting discipline; build a full model on a real listed company (e.g. from screener.in data)

Resources:

- **Free:** Aswath Damodaran (NYU) — his valuation lectures and datasets (ERP, betas by industry) are the global gold standard, free on his site/YouTube. Corporate Finance Institute free articles. `screener.in` for Indian company financials.
- **Paid:** Wall Street Prep / Breaking Into Wall Street (BIWS) DCF modeling courses; CFI's **FMVA** certification (globally portable, covers DCF end-to-end); Wall Street Oasis modeling tests for practice.
- **India-relevant:** CFA Level II (Equity) covers FCFF/FCFE rigorously; NISM Research Analyst certification touches valuation.

Quick-reference

```
FCFF = EBIT×(1−t) + D&A − Capex − ΔNWC
Ke   = Rf + β×ERP                        (CAPM)
Kd,at= Kd×(1−t)
WACC = E/(E+D)×Ke + D/(E+D)×Kd,at        (market weights)
βL   = βU×(1 + (1−t)×D/E)                (Hamada re-lever)
DF   = 1/(1+WACC)^n   [mid-year: n−0.5]
TV_Gordon = FCFF_N×(1+g)/(WACC−g)        → then ×DF_N
TV_Exit   = EBITDA_N × multiple          → then ×DF_N
EV   = Σ(FCFF×DF) + TV×DF_N
Equity = EV − NetDebt − Minority − Pref + Associates
Price = Equity / Diluted shares
```

Input	Sensible range (India)	Developed markets
Risk-free (Rf)	6.5–7.5% (10-yr G-sec)	3.5–4.5% (US 10-yr)
Equity risk premium	7–9%	4.5–5.5%
Terminal growth g	4–6%	2–3%
WACC (typical)	11–15%	7–10%
Terminal value as % of EV	60–80% — always stress-test it	

Golden rules: g < WACC always. Discount the TV. FCFF↔WACC, FCFE↔Ke. Every assumption in a blue input cell. Ship the sensitivity table.

Sensitivity, Scenario & Data Tables

What it is & where it's used

Sensitivity and scenario analysis answer the question every decision-maker actually asks: *"What if I'm wrong?"* Your model spits out one number — an NPV of ₹4.2 crore, an EPS of ₹18.50, a breakeven of 62,000 units. But that number rests on assumptions (growth, margin, discount rate) that are just educated guesses. Sensitivity analysis measures how much the output moves when an input moves. Scenario analysis bundles several inputs into named cases (Base / Best / Worst). Together they turn a fragile point-estimate into a defensible range.

Where it shows up on the job:

Role	Use
FP&A / Corporate finance	Budget scenarios, revenue sensitivity, board decks
Investment banking / PE	DCF valuation sensitivity, LBO returns grids
Equity research	Target-price football fields, EPS scenarios
Credit / Lending	Stress-testing DSCR and interest-coverage covenants
Project finance / Capex	IRR sensitivity to capacity, tariff, delay
Treasury	FX and rate impact on cash flow

The four core Excel tools: **Data Tables** (one/two-variable grids), **Scenario Manager** (named input sets), **Goal Seek** (reverse-solve one input for a target), and **Tornado charts** (rank which drivers matter most).

The gap: why companies want this (and college didn't teach it)

An MBA finance course teaches you to *calculate* NPV and IRR. It stops there. It hands you clean inputs and asks for one answer. Industry does the opposite: nobody trusts a single answer, and half your inputs are contested in the meeting.

The specific gaps employers see in freshers:

- You built a beautiful DCF, but when the CFO says *"show me NPV if EBITDA margin drops 200 bps and WACC is 13%"* you rebuild the whole thing manually instead of reading it off a two-variable data table in three seconds.
- You hard-coded assumptions inside formulas, so nothing is switchable and no sensitivity is possible.
- You confuse a **scenario** (many inputs change together, e.g. a recession) with a **sensitivity** (one input flexes, all else held). They answer different questions.
- You don't know which of your 30 assumptions actually moves the answer — so you spend hours refining an input the output barely cares about.

Companies pay for the person who can say: *"NPV is positive across every scenario except a demand shock below 45,000 units; the two drivers that matter are price realisation and WACC — everything else is noise."* That sentence is the job.

What "proficient" looks like

A job-ready person can, unaided:

1. **Structure a model for sensitivity** — every assumption in a labelled input cell, output linked by formula, nothing hard-coded.
2. Build a **one-variable and two-variable data table** correctly (including the notorious "top-left cell must reference the output" rule).
3. Set up **Scenario Manager** with 3+ named scenarios and produce a **Scenario Summary** report.
4. Use **Goal Seek** to reverse-solve (e.g. "what price gives ₹0 NPV?") and know its limits (one changing cell, one target).
5. Build a **tornado chart** to rank drivers by impact.
6. Read a sensitivity grid and **state a decision**, not just present numbers.
7. Know that data tables are **volatile** (recalc-heavy) and how to switch to *Automatic Except Data Tables* on big models.

Hands-on: how to actually do it

Setup: a switchable model

Never write `=B5*1.1`. Put the 1.1 (growth) in its own cell and reference it. Assume this layout:

```
      A                        B
1   Units sold               50,000
2   Price per unit (₹)          800
3   Variable cost/unit (₹)      480
4   Fixed cost (₹)        1,20,00,000
5   Tax rate                    25%
6
7   Revenue                  =B1*B2
8   Contribution          =(B2-B3)*B1
9   PBT                      =B8-B4
10  PAT                  =B9*(1-B5)
```

`B10` (PAT) is our output. Say it computes ₹78,00,000.

One-variable data table

Question: how does PAT change as **Units sold** varies from 30,000 to 70,000?

1. In a column, list the input values (say `D2:D6` = 30000, 40000, 50000, 60000, 70000).
2. In the cell **one row up and one column right** of the first value (`E1`), reference the output: `=B10`.
3. Select the rectangle `D1:E6`.
4. **Data ▸ What-If Analysis ▸ Data Table**.
5. Because inputs run down a column, put the input cell in the **Column input cell** box: `$B$1`. Leave Row input blank. OK.

Excel fills `E2:E6` with PAT at each unit level. It substitutes each value into `B1` and reads `B10`.

Two-variable data table

Question: PAT across combinations of **Units (rows)** and **Price (columns)**.

```
        D           E        F        G        H
1     =B10        750      800      850      900     ← prices across the top
2    30,000
3    40,000
4    50,000
5    60,000
6    70,000
```

- **Top-left corner cell (`D1`) MUST reference the output**: `=B10`. This is the #1 error.
- Units go down column D; prices go across row 1.
- Select `D1:H6` ▸ **What-If Analysis** ▸ **Data Table**.
- **Row input cell** = `$B$2` (price is along the row). **Column input cell** = `$B$1` (units down the column). OK.

You now have a 5×4 grid of PAT for every unit/price combo.

Goal Seek — reverse-solve

Question: what **price** makes PAT exactly ₹1,00,00,000?

Data ▸ **What-If Analysis** ▸ **Goal Seek** - Set cell: `$B$10` - To value: `10000000` - By changing cell: `$B$2`

Excel iterates and drops the required price into `B2`. Classic uses: breakeven price, required volume for a target profit, the interest rate that zeroes an NPV (a manual IRR).

Scenario Manager — named cases

Data ▸ **What-If Analysis** ▸ **Scenario Manager** ▸ **Add**. Changing cells: `$B$1,$B$2,$B$3`.

Scenario	Units (B1)	Price (B2)	VC/unit (B3)
Base	50,000	800	480
Best	65,000	850	460
Worst	38,000	740	510

Add each set of values. Then **Summary** ▸ **Scenario summary**, result cell `$B$10`. Excel builds a clean comparison sheet of PAT across all three — perfect for a board slide.

The formula alternative (more robust than data tables)

For a clean sensitivity grid without volatile data tables, use a corner-output cell plus direct recompute, or in modern Excel spill it:

```
=LET(u, SEQUENCE(1,,30000,10000),
     p, SEQUENCE(5,,750,50),
     ((800 - 480) ... )   // build the PAT expression across u and p
)
```

In practice most desks still use data tables — but know that `INDEX` / `CHOOSE` scenario switches are more auditable:

```
Active scenario in B12 (1/2/3):
Units  =CHOOSE($B$12, 50000, 65000, 38000)
Price  =CHOOSE($B$12, 800, 850, 740)
```

Flip `B12` and the whole model swings. This is how professional models toggle scenarios.

Python cross-check (for larger analyses)

```
import numpy as np, pandas as pd
units = np.arange(30000, 70001, 10000)
prices = np.arange(750, 901, 50)
def pat(u, p, vc=480, fc=1.2e7, tax=0.25):
    return ((p - vc) * u - fc) * (1 - tax)
grid = pd.DataFrame({p: pat(units, p) for p in prices}, index=units)
print(grid.round(0))
```

Worked example / mini-project

Capex decision: should a firm buy a ₹2 crore packaging machine?

Assumptions (all in input cells):

Input	Cell	Base
Initial outlay (₹)	B1	2,00,00,000
Annual units	B2	5,00,000
Contribution/unit (₹)	B3	40
Annual fixed cost (₹)	B4	60,00,000
Life (years)	B5	5
WACC	B6	12%
Tax rate	B7	25%

Annual after-tax cash flow (ignoring depreciation tax shield for simplicity):

```
Annual CF  =((B2*B3)-B4)*(1-B7)     → ((5,00,000*40)-60,00,000)*0.75 = ₹1,05,00,000
NPV        =-B1 + NPV(B6, <5 equal annual CFs>)
```

`NPV(12%, five years of ₹1.05 cr) − ₹2 cr = ₹3.78 cr − 2.00 cr = ₹1.78 cr`. Positive — accept, at base case.

Now stress it. Two-variable data table: WACC (rows) × Units (columns), output = NPV.

NPV (₹ cr)	3,50,000	4,25,000	5,00,000	5,75,000
10%	0.24	1.30	2.36	3.42
12%	0.05	1.06	2.08	3.10
14%	-0.12	0.85	1.82	2.79
16%	-0.28	0.65	1.58	2.51

Decision read-off: NPV stays positive across almost the entire grid. It only turns negative if volume falls to ~3,50,000 units *and* WACC hits 14%+ simultaneously — a genuine double-shock. The project is robust; the binding risk is a demand collapse, not the cost of capital.

Goal Seek the breakeven: Set NPV = 0 by changing Units → ~3,35,000 units. So the firm has a ~33% volume cushion below base. That single sentence is what goes to the investment committee.

Tornado (one-at-a-time ±10%): flex each input ±10%, record NPV swing, sort descending:

Driver	NPV low	NPV high	Swing (₹ cr)
Contribution/unit	1.19	2.36	1.17
Units	1.24	2.31	1.07
Fixed cost	1.66	1.89	0.23
WACC	1.70	1.86	0.16

Plot as a horizontal bar chart sorted widest-at-top → a tornado. Contribution and volume dominate; fixed cost and WACC barely register. Focus diligence there.

How it's tested

Interview questions - "Difference between sensitivity and scenario analysis?" (one input vs many inputs moving together.) - "In a two-variable data table, what goes in the top-left cell?" (a reference to the output.) - "How would you find the price at which NPV = 0?" (Goal Seek / a data table.) - "Your NPV is very sensitive to WACC — what does that tell you?" (long-duration cash flows; valuation risk; justify the discount rate carefully.) - "Why is Data Table sometimes disabled or slow?" (it's a volatile array; on big models set Formulas ▸ Calculation ▸ *Automatic Except Data Tables*.)

Practical / assessment tests - **Timed Excel test (30–45 min):** given a P&L or DCF, "build a two-variable data table of NPV vs growth and margin, add a Base/Best/Worst scenario summary, and state your recommendation." They watch whether you hard-code, whether the top-left cell is right, and whether you actually conclude. - **Case study:** "Here's a project; tell us how sensitive the return is and what would kill it."

They want the tornado insight, not a data dump. - **Live screen-share:** they change an assumption and check your model updates end-to-end (proving nothing is hard-coded).

Common mistakes & how pros avoid them

Mistake	Fix
Hard-coding assumptions in formulas	Every assumption in its own labelled input cell
Top-left cell of a 2-var table has a number, not `=output`	Always point it at the output cell
Swapping Row and Column input cells	Row input = the variable running *across*; Column input = the one running *down*
Data table input cells on a different sheet	Excel forbids it — inputs must be on the same sheet as the table
Confusing scenario with sensitivity	Scenario = many inputs change together; sensitivity = one at a time
Model crawls after adding tables	Set *Automatic Except Data Tables*; press F9 to recalc deliberately
Presenting the grid with no conclusion	End with one sentence: what's the decision and what's the binding risk
Tornado with inconsistent ±ranges	Flex every driver by the *same* % (or same std-dev) so bars are comparable
Goal Seek won't converge	Give a sensible start value; it needs a monotonic, solvable relationship

Learn-it roadmap & resources

Time to proficiency: 1–2 weeks of focused practice if you already know NPV/IRR. A weekend gets you functional; real fluency comes from building 5–6 of your own models.

Week	Focus
Days 1–2	Switchable model structure; one-variable data table
Days 3–4	Two-variable tables; Goal Seek
Day 5	Scenario Manager + Summary report
Days 6–7	Tornado chart; write decision memos from grids

Resources - *Free:* Corporate Finance Institute (CFI) free Excel articles; Microsoft's "What-If Analysis" docs; ExcelJet (searchable formula reference); Aswath Damodaran's spreadsheets (nyu.edu/~adamodar) — real DCFs with live sensitivity. - *Paid / certification:* CFI's **FMVA** (Financial Modeling & Valuation Analyst) — heavy on scenarios and sensitivity; Wall Street Prep / BIWS modeling courses. Any of these signals to Indian recruiters (Big 4, IB, corporate FP&A) that you can model, not just calculate. - *Practice:* rebuild a listed company's DCF from its annual report and add a WACC × growth grid — the single best portfolio piece for a finance interview.

Quick-reference

Tool	Path	Answers
One-variable data table	Data ▸ What-If ▸ Data Table (fill Column *or* Row input)	Output vs one input
Two-variable data table	Same; fill **both** Row & Column input; top-left = `=output`	Output vs two inputs
Goal Seek	Data ▸ What-If ▸ Goal Seek	Reverse-solve one input for a target
Scenario Manager	Data ▸ What-If ▸ Scenario Manager	Named Base/Best/Worst cases
Scenario switch (formula)	`=CHOOSE($B$12, base, best, worst)`	Auditable toggle
Tornado chart	±X% each driver → sorted bar chart	Which drivers matter most

Golden rules - Top-left cell of a two-variable table = `=` output cell. - Row input = variable across the top; Column input = variable down the side. - Data table inputs must be on the **same sheet**. - Data tables are volatile → *Automatic Except Data Tables* on large models. - Never hard-code an assumption. Always end with a decision, not a grid.

Forecasting & budgeting models

What it is & where it's used

A forecasting or budgeting model is a spreadsheet that turns **assumptions about the future** into projected P&L, and often cash flow and balance sheet. You feed it *drivers* — units sold, price, headcount, conversion rate, seasonality — and it computes revenue, cost, and margin forward in time (usually 12–36 months).

Where it lives:

- **FP&A / finance business partner** — builds the annual operating plan (AOP) and monthly rolling forecast.
- **Startup finance / founder's office** — revenue build for the board deck and runway model.
- **Corporate finance / treasury** — cash forecast, working-capital planning.
- **Investment banking / equity research** — the projection block in a DCF or three-statement model.
- **Accounts / controllership** — budget-vs-actual (BvA) variance packs each month-end.

Almost every "analyst" job in India that pays above a data-entry salary expects you to *own* a forecast, defend the numbers, and update it monthly without breaking it.

The gap: why companies want this (and college didn't teach it)

College teaches you to **extrapolate a trend** — "revenue grew 15% last year, so grow it 15%." That is a straight-line forecast and every good FP&A lead will reject it in an interview.

The industry gap this chapter closes:

College taught	Industry actually wants
Grow revenue by a %	**Driver-based** build: `Volume × Price`, `Traffic × Conv × AOV`
One annual number	**Monthly granularity** with seasonality
A static budget locked in April	A **rolling forecast** re-cut every month
"Forecast = prediction"	Forecast = a **structured set of assumptions you can defend and flex**
Hardcoded numbers in cells	Clean **assumptions tab**, blue inputs, black formulas, no hardcodes in calc cells

The commercial reason companies care: a driver-based model lets leadership ask *"what if we hire 3 fewer salespeople?"* and get an answer in 10 seconds. A trend line can't do that. That flex-ability is the paid skill.

What "proficient" looks like

A job-ready person can, unaided:

1. Separate **inputs (assumptions)** from **calculations** from **outputs** on distinct tabs, with a consistent colour convention.
2. Build a **revenue driver tree** — not one growth %, but the 2–4 levers that actually move revenue.
3. Apply **monthly seasonality** using a seasonality index (12 factors that average to 1.0).
4. Build a **12-month budget** that footsends (rows and columns cross-check to the same total).

5. Convert that budget into a **rolling forecast**: actuals replace forecast as months close, and a new month is added at the far end.
6. Build a **budget-vs-actual variance** view with % variance and a favourable/adverse flag.
7. Never hardcode a number inside a formula; every input traces to the assumptions tab.

Hands-on: how to actually do it

1. Driver-based revenue build

Say revenue = `Units × Price`. Put units and price on the assumptions tab, reference them in the calc.

For a subscription/SaaS or services build, use a **customer roll-forward**:

```
Opening customers  + New adds  − Churn  = Closing customers
Revenue = Avg customers in month × ARPU
```

Excel (row per month, column B = assumptions):

```
' Closing customers this month
=Opening + New_Adds − (Opening * Churn_Rate)

' New adds from marketing spend
=Marketing_Spend / CAC

' Revenue
=AVERAGE(Opening, Closing) * ARPU
```

For an e-commerce / retail build:

```
Revenue = Sessions * Conversion_Rate * Average_Order_Value
=B$4 * B$5 * B$6
```

Lock the assumption rows with `$` on the row (`B$4`) so you can drag across all 12 months.

2. Seasonality with an index

Build 12 seasonality factors that average to exactly 1.0. In India, festive months (Oct–Nov, Diwali) and year-end (Mar) usually spike.

```
' Seasonality index row (Jan..Dec), must average 1
Jan 0.85  Feb 0.80  Mar 1.20  ...  Oct 1.25  Nov 1.30  ...
' Check it averages 1.0:
=AVERAGE(B10:M10)   → should return 1.000

' Seasonalised monthly revenue
=Annual_Revenue/12 * Seasonality_Factor
```

Derive the index from history:

```
Factor_for_month = Month_actual / AVERAGE(all 12 months' actual)
```

3. Growth over time (driver, not hardcode)

```
' This month's units = last month's units grown at monthly rate
=C_prev_units * (1 + MoM_growth)

' Convert an annual growth target to monthly:
=(1 + Annual_Growth)^(1/12) - 1
```

4. Rolling forecast switch — actual vs forecast

Add an "Actuals through" date on the assumptions tab. Each month cell picks actual if the month has closed, else forecast:

```
=IF(EOMONTH(MonthDate,0) ≤ Actuals_Through_Date, Actual_Value, Forecast_Value)
```

5. Budget-vs-actual variance

```
Variance      =Actual - Budget
Variance %    =IFERROR(Actual/Budget - 1, "")
Flag          =IF(Metric_Type="Cost", IF(Actual>Budget,"Adverse","Favourable"),
                                      IF(Actual<Budget,"Adverse","Favourable"))
```

6. Python — quick statistical baseline (sanity check your driver model)

```
import pandas as pd
# monthly revenue history
s = df['revenue']
# 3-month moving average forecast
ma = s.rolling(3).mean().iloc[-1]
# seasonal-naive: same month last year
seasonal = s.shift(12).iloc[-1]
print(ma, seasonal)
```

For a proper seasonal forecast:

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
model = ExponentialSmoothing(s, trend='add', seasonal='mul',
                             seasonal_periods=12).fit()
forecast = model.forecast(12)
```

Use this to *challenge* the business build, not replace it.

Worked example / mini-project

"D2C skincare brand — FY27 budget." Reproduce this in a fresh workbook.

Assumptions tab:

Driver	Value
Monthly sessions (Apr)	4,00,000
Session MoM growth	3%
Conversion rate	2.2%
Average order value	₹950
Gross margin	62%
Marketing (% of revenue)	18%
Fixed opex / month	₹35,00,000

Seasonality index (Apr…Mar): 0.95, 0.90, 0.90, 1.00, 1.05, 1.10, **1.30, 1.25**, 1.05, 0.95, 0.90, 1.15 (avg = 1.0). The Oct–Nov spike is Diwali.

Monthly revenue formula:

```
Revenue = Sessions_this_month * Conv * AOV * Seasonality_Factor
```

Sample first quarter (rounded):

Metric	Apr	May	Jun
Sessions	4,00,000	4,12,000	4,24,360
Base revenue (₹)	83,60,000	86,10,800	88,69,124
× Seasonality	0.95	0.90	0.90
Net revenue (₹)	**79,42,000**	**77,49,720**	**79,82,212**
COGS @ 38%	30,17,960	29,44,894	30,33,241
Gross profit (₹)	49,24,040	48,04,826	49,48,971
Marketing @ 18%	14,29,560	13,94,950	14,36,798
Fixed opex	35,00,000	35,00,000	35,00,000
EBITDA (₹)	**-5,52,120**	**-1,90,124**	**12,173**

Now add the **rolling forecast layer**: set `Actuals_Through_Date = 31-May-26`. Suppose actual May revenue came in at ₹82,00,000 (beat). Replace May's forecast with actual via the `IF(EOMONTH ... )` switch, then re-forecast Jun–Mar off the *new* higher base. Board question you can now answer instantly: "*May beat by ₹4.5L — does that flow through the year?*" Yes, because Jun sessions grow off May.

Variance pack for May:

Line	Budget	Actual	Var	Var %	Flag
Revenue	77,49,720	82,00,000	4,50,280	+5.8%	Favourable
Marketing	13,94,950	15,10,000	1,15,050	+8.2%	Adverse

How it's tested

Interview questions:

- "How would you forecast revenue for a business you know nothing about?" → answer with a driver tree, not a growth %.
- "Difference between a budget and a rolling forecast?" → budget is fixed annual target; rolling forecast is continuously re-cut (e.g. always 12 months ahead).
- "Your forecast beat in Q1 — how do you decide whether to raise the full-year number?" → tests whether you understand run-rate vs one-off.
- "What's a seasonality index and how do you build one?"

Practical / take-home tests companies actually give:

- A **timed 60–90 min Excel case**: raw monthly history in one tab, "build a 12-month driver-based forecast with seasonality and a BvA view." Graded on structure (assumptions separated), no hardcodes, and whether it *flexes* when they change an input live.
- A **"break my model"** screen: they hand you a model, change one assumption, and check your cells all update (catches hardcoders).
- FP&A rounds often include a **live case**: "walk me through your model and defend the marketing-as-%-of-revenue assumption."

Common mistakes & how pros avoid them

Mistake	Pro fix
Hardcoding numbers inside formulas	Every input on assumptions tab; calc cells are pure formulas. Blue = input, black = formula
One growth % for all revenue	Break into 2–4 real drivers
Seasonality factors that don't average to 1.0	Add a `=AVERAGE()` check cell that must read 1.000
Forecast doesn't foot	Cross-check: sum of months = annual; sum of segments = total
Rebuilding the whole model each month	Build the `IF(EOMONTH≤Actuals_Through)` switch once
No scenario capability	Keep a single "case" toggle driving assumptions via `CHOOSE` / `INDEX`
Circular references from interest-on-cash	Use a circularity switch or iterative calc deliberately, not by accident
Over-precision (forecasting to the rupee)	Forecast drivers, round outputs; defend ranges not points

Learn-it roadmap & resources

Time to proficiency: ~4–6 weeks part-time if you already know Excel basics.

- **Week 1:** Model structure — separate tabs, colour convention, no hardcodes. Rebuild the worked example above.
- **Week 2:** Driver trees for 3 business types (SaaS, retail, services). Build each from scratch.
- **Week 3:** Seasonality index from real history; MoM vs annual growth conversion.
- **Week 4:** Rolling forecast switch + BvA variance pack.
- **Weeks 5–6:** Add scenario toggle, connect to a simple three-statement model, do a timed 90-min case against a clock.

Resources:

- *Financial Modeling* — CFI (Corporate Finance Institute) FMVA track (paid, well-recognised in India).
- Wall Street Prep / Breaking Into Wall Street modeling courses (paid).
- Free: Aswath Damodaran's forecasting lectures (YouTube); Microsoft's own XLOOKUP/dynamic-array docs.
- Certification: **FMVA** (CFI) is the most portable for FP&A roles; for pure Excel, the **Microsoft Excel Expert (MO-201)** cert signals hard skills.

Quick-reference

```
' Driver revenue (lock assumption row)
=Sessions * B$5_Conv * B$6_AOV * B$10_Seasonality

' Annual growth → monthly growth
=(1+Annual_Growth)^(1/12)-1

' Grow off prior month
=Prev * (1 + MoM_Growth)

' Customer roll-forward
Closing = Opening + Adds - Opening*Churn

' Seasonality check (must = 1.000)
=AVERAGE(seasonality_range)

' Rolling forecast switch
=IF(EOMONTH(MonthDate,0)≤Actuals_Through, Actual, Forecast)

' Variance & flag
Var    =Actual-Budget
Var%   =IFERROR(Actual/Budget-1,"")
Flag   =IF(Actual>Budget,"Adverse","Favourable")   ' for costs
```

Concept	One-liner
Budget	Fixed annual target, locked at start of year
Rolling forecast	Re-cut monthly, always N months ahead
Driver-based	Revenue = drivers multiplied, not a single %
Seasonality index	12 factors averaging 1.0 applied to base
BvA	Budget vs Actual, with variance % and F/A flag
Colour code	Blue = input, black = formula, green = link

LBO & Returns Model Basics

What it is & where it's used

A Leveraged Buyout (LBO) is the acquisition of a company funded mostly with **debt** and a slice of **equity**. A private-equity (PE) fund buys a business, uses the target's own cash flows to repay that debt over 4-6 years, and sells at exit. Because debt is paid down and (hopefully) the business grows, the equity slice multiplies in value. The LBO model is the spreadsheet that proves the deal makes money — it links **Sources & Uses**, a **debt schedule with a cash sweep**, and an **equity returns** block (IRR and MOIC).

Who builds these: - **PE / buyout funds** (Blackstone, KKR, in India: ChrysCapital, Kedaara, Multiples, True North) — every deal is an LBO model. - **Investment banking** (Leveraged Finance, Sponsors coverage, M&A) — banks size the debt and stress-test it. - **Credit funds / NBFC structured finance** — lend into the deal, model the debt from the lender's side. - **Corp-dev / strategy** teams doing acquisitions with acquisition financing. - **Equity research / valuation** — an LBO gives a "floor" valuation (what a financial buyer would pay).

It is the single most common technical case in PE and LevFin interviews.

The gap: why companies want this (and college didn't teach it)

MBA finance teaches NPV, WACC, and CAPM — a *cost of capital* mindset. LBO is a *cash-and-debt-paydown* mindset. The gap is specific:

- College values a firm with one discount rate. PE cares about **how the capital structure changes year by year** as debt amortizes.
- College treats leverage as a WACC input. In an LBO, leverage is the **engine of returns** — you must build the debt schedule that mechanically sweeps free cash into repayment.
- Textbooks skip the **circularity**: interest depends on debt balance, which depends on cash sweep, which depends on cash-after-interest. Real models handle this with iterative calc or an interest-on-opening-balance shortcut.
- Nobody teaches **Sources & Uses** or how a returns bridge decomposes IRR into deleveraging vs. EBITDA growth vs. multiple expansion.

Employers pay for someone who can build the whole thing in 45 minutes without a template.

What "proficient" looks like

A job-ready person can, unaided:

1. Build a **Sources & Uses** table that balances to the penny.
2. Compute entry Enterprise Value from an **EV/EBITDA** multiple and back into equity cheque.
3. Build a **debt schedule** for a Term Loan (mandatory amortization + cash sweep) and a revolver.
4. Project a simple **FCF-before-debt** line and route it through the sweep.
5. Compute **exit equity value**, then **MOIC** (= exit equity / entry equity) and **IRR** using `=IRR()` or `=XIRR()`.
6. Explain a **returns bridge**: how much of the gain came from paying down debt vs. growing EBITDA vs. selling at a higher multiple.

7. Run sensitivities (entry multiple, exit multiple, leverage) with a **Data Table**.

Hands-on: how to actually do it

Step 1 — Sources & Uses

Uses = what you pay for. Sources = how you fund it. They must equal.

Sources	Rs cr	Uses	Rs cr
Term Loan (Debt)	= `D_ebitda*4.0`	Purchase Enterprise Value	= `EntryEBITDA*EntryMult`
Sponsor Equity (plug)	=Total Uses − Debt	Transaction fees (2%)	= `EV*0.02`
Total Sources	=SUM	**Total Uses**	=SUM

Excel — say entry EBITDA in `B3`, entry multiple in `B4`, leverage (Debt/EBITDA) in `B5`:

```
EV              =B3*B4
Debt            =B3*B5
Fees            =EV*0.02
Total Uses      =EV+Fees
Equity (plug)   =Total_Uses - Debt      ' the sponsor cheque
```

Step 2 — Project EBITDA and FCF

```
Year               1        2        3        4        5
EBITDA          =prior*(1+g)   ...
Less: D&A       =-EBITDA*0.06
EBIT            =EBITDA+D&A
Less: Interest  =-Opening_Debt*rate      ' link to debt schedule
EBT             =EBIT+Interest
Less: Tax@25%   =-MAX(EBT,0)*0.25
Net Income      =EBT+Tax
Add: D&A        =-D&A line
Less: Capex     =-EBITDA*0.05
Less: ΔNWC      =-Revenue_change*0.10
FCF (pre-sweep) =NI + D&A + Capex + ΔNWC
```

Step 3 — Debt schedule with cash sweep

The sweep: after mandatory amortization, any leftover cash **prepays** the term loan.

```
Opening Debt          =prior Closing Debt
Mandatory amort (5%)  =-MIN(Opening, Original_TL*0.05)
Cash avail for sweep  =FCF_pre_sweep + Mandatory_amort   ' amort already negative
Optional sweep        =-MIN(MAX(Cash_avail,0), Opening+Mandatory_amort)
Closing Debt          =Opening + Mandatory + Optional_sweep
Interest expense      =Opening_Debt * rate               ' opening-balance avoids circularity
```

Using **opening balance** for interest breaks the circular reference — the standard interview shortcut. (Production models use average balance + enable iterative calc: File > Options > Formulas > Enable iterative calculation, max 100.)

Step 4 — Exit and returns

```
Exit EBITDA       = Year-5 EBITDA
Exit EV           = Exit_EBITDA * Exit_Multiple
Exit Net Debt     = Year-5 Closing Debt - Cash
Exit Equity       = Exit_EV - Exit_Net_Debt
MOIC              = Exit_Equity / Entry_Equity
IRR               = (MOIC)^(1/Years) - 1        ' single in/out shortcut
```

In Excel with actual dated cash flows (equity out at entry, in at exit):

```
=IRR({-Entry_Equity, 0, 0, 0, 0, Exit_Equity})
=XIRR(values_range, dates_range)     ' when dates are irregular
=MOIC^(1/5)-1                          ' quick CAGR check
```

Worked example / mini-project

Target: an Indian mid-market auto-components firm. Entry EBITDA Rs 100 cr, entry multiple 8.0x, EBITDA grows 8%/yr, hold 5 years, exit at 8.0x (flat — conservative). Leverage 4.0x. Interest 11%. Tax 25%. D&A 6% of EBITDA, Capex 5%, ΔNWC 10% of revenue growth (assume EBITDA≈EBIT proxy for simplicity, revenue ≈ 2x EBITDA).

Sources & Uses (Rs cr):

Uses	Rs cr	Sources	Rs cr
Entry EV (100 × 8.0)	800	Term Loan (4.0x)	400
Fees (2%)	16	Sponsor Equity	416
Total	**816**	**Total**	**816**

Debt paydown (opening-balance interest, 5% mandatory amort, full sweep):

Year	EBITDA	Interest@11%	Tax@25% approx	FCF pre-sweep	Opening Debt	Closing Debt
1	108	44.0	~13	~58	400	342
2	116.6	37.6	~16	~66	342	276
3	126.0	30.4	~19	~72	276	204
4	136.0	22.4	~22	~80	204	124
5	146.9	13.6	~26	~88	124	36

(FCF ≈ EBITDA − interest − tax − capex − ΔNWC; numbers rounded to show the mechanic, not to the last rupee.)

Exit:

```
Exit EBITDA      = 146.9
Exit EV          = 146.9 × 8.0 = 1,175
Exit Net Debt    = 36
Exit Equity      = 1,175 − 36 = 1,139
Entry Equity     = 416
MOIC             = 1,139 / 416 = 2.74x
IRR              = 2.74^(1/5) − 1 = 22.3%
```

Returns bridge — where did the 2.74x come from? With a *flat* multiple, every rupee came from **EBITDA growth** (100 → 147) and **deleveraging** (net debt 400 → 36). Zero from multiple expansion. That is a *clean, defensible* deal — you don't need to sell higher than you bought.

Reproduce it: put assumptions in a block, build the 5-year strip, and add a two-way **Data Table** (Data > What-If Analysis > Data Table) with exit multiple across the top (7.0x–9.0x) and leverage down the side (3.0x–5.0x) referencing the IRR cell. You'll see IRR swing roughly 15%–30%.

How it's tested

Interview questions: - "Walk me through an LBO." (Sources & Uses → buy → pay down debt with FCF → sell → equity multiplies.) - "What are the 3 drivers of LBO returns?" (Deleveraging, EBITDA growth, multiple expansion.) - "If I double the leverage, what happens to IRR?" (Higher — smaller equity cheque — but higher risk / interest burden.) - "Why use opening-balance interest?" (Avoids circular reference in a timed test.) - "MOIC of 2.5x over 5 years ≈ what IRR?" (~20%; know 2x/5yr≈15%, 3x/5yr≈25%.)

Practical tests companies give: - **Paper LBO** (no computer, 5 min): entry 5x, exit 5x, given growth and paydown — compute IRR mentally. Extremely common at PE first rounds (ChrysCapital, Everstone, bulge-bracket LevFin). - **Timed Excel LBO** (30-60 min): build the full model from a one-page CIM summary. Judged on: does S&U balance, is the debt schedule right, no hardcodes, clean IRR. - **Take-home case**: full model + 2-slide recommendation over a weekend.

Common mistakes & how pros avoid them

Mistake	Fix
Sources ≠ Uses	Always make **equity the plug** = Total Uses − Debt. Never hardcode equity.
Forgetting **fees** in Uses	Add transaction + financing fees; they raise the equity cheque.
Circular-reference `#REF` spiral	Use **opening-balance** interest, or enable iterative calc deliberately.
Sweeping debt **below zero**	Wrap sweep in `=MIN(cash, opening_balance)` so it can't over-repay.
Confusing EV and equity at exit	Exit **equity** = Exit EV − **net debt**. Subtract debt, add cash.
MOIC vs IRR confusion	MOIC ignores time; IRR is annualized. A 3x over 10 years (12% IRR) is worse than 2x over 3 years (26%).
Assuming multiple expansion	Model **flat exit multiple** as base case. Expansion is a bonus, never the thesis.
Ignoring a **cash flow sanity check**	FCF must stay positive; if interest > FCF the deal is over-levered.

Learn-it roadmap & resources

Time to proficiency: 2-3 weeks of focused practice to build one unaided; 6-8 weeks to be interview-fast (including paper LBOs).

Week	Focus
1	Sources & Uses + entry/exit mechanics. Build the S&U 10 times.
2	Debt schedule + cash sweep + circularity. This is the hard part.
3	Returns bridge, sensitivities, paper LBOs on a stopwatch.

Resources: - **Free:** Wall Street Prep / Macabacus free LBO tutorials; Aswath Damodaran (NYU) sessions on YouTube for the valuation base; Corporate Finance Institute free templates. - **Paid:** Breaking Into Wall Street (BIWS) "LBO Modeling" course; Wall Street Prep Premium; Rossum / Financial Edge for the India/UK market. - **Certification:** CFA (Level II corporate finance / equity), or FMVA (CFI) which includes an LBO module. For India PE roles, a strong self-built model beats any certificate. - **Practice:** rebuild any listed mid-cap as an LBO — pull EBITDA from screener.in, assume 4x leverage, model 5 years.

Quick-reference

```
EV              = EBITDA × Entry Multiple
Equity (entry)  = Total Uses − Debt          [equity is the plug]
Total Uses      = EV + Fees
Interest        = Opening Debt × rate         [avoids circularity]
Sweep           = MIN(FCF_after_mandatory, Opening Debt)
Closing Debt    = Opening − Mandatory − Sweep
Exit Equity     = Exit EBITDA × Exit Mult − Net Debt
MOIC            = Exit Equity / Entry Equity
IRR             = MOIC^(1/years) − 1          [single in/out]
```

Quick IRR ↔ MOIC (5-year hold)	MOIC	IRR
	1.5x	~8%
	2.0x	~15%
	2.5x	~20%
	3.0x	~25%
	4.0x	~32%

Three return drivers: (1) Deleveraging — debt paid by FCF. (2) EBITDA growth — organic + margin. (3) Multiple expansion — sell higher than bought (model flat; treat as upside).

Key Excel: `=IRR()`, `=XIRR(values,dates)`, `=MIN()` / `=MAX()` for the sweep, Data > What-If > Data Table for sensitivities, File > Options > Formulas > Enable iterative calculation for average-balance interest.

Model best-practice & error-proofing

What it is & where it's used

A financial model is code written in Excel. Like code, it can be readable and auditable — or a tangle nobody dares touch. "Model best-practice" is the set of conventions professionals use so that any reviewer can open your workbook, understand which cells are assumptions, trace every number to its source, and trust the totals. "Error-proofing" is the layer of self-checks that scream when the model breaks.

This matters in every role that hands a spreadsheet to someone senior: FP&A analysts building the annual budget, investment banking / PE associates running LBO and DCF models, corporate finance teams building three-statement models, credit analysts sizing debt, and even accounts/tax teams preparing schedules that feed the return. In India, a Big 4 valuation team, a startup's finance team building an investor model, or a treasury desk reconciling cash — all of them will judge you on whether your model is disciplined, not just whether the answer is "right" today.

The skill is invisible when done well and painfully visible when skipped. A model that balances by luck is worthless; a model whose check row flashes red the moment an input is fat-fingered is worth a promotion.

The gap: why companies want this (and college didn't teach it)

Your MBA taught you *what* a DCF is: project free cash flows, discount at WACC, sum to enterprise value. It graded you on the answer. It never graded you on whether the cell was a formula or a typed-in number, whether a stranger could audit it, or whether it would survive being reused next quarter.

The industry reality is the opposite. Nobody builds a model once. It gets reopened, forwarded, stress-tested in a live meeting, and inherited by the next analyst when you move on. The single most common reason a deal team distrusts a junior's work is not a wrong assumption — it's a hardcoded number buried inside a formula (`=E10*1.08` where did 1.08 come from?), a broken link, or a balance sheet that doesn't balance and no check to catch it.

Colleges teach models as disposable homework. Employers treat them as durable, shared infrastructure. That gap — from "get the answer" to "build something another human can trust, audit, and extend" — is exactly what this chapter closes, and it is disproportionately rewarded because so few freshers have it.

What "proficient" looks like

A job-ready person, handed a blank workbook, will unaided:

- **Colour-code** every cell by type without being asked: blue = hardcoded input, black = formula, green = links from other sheets, red = external workbook link.
- Keep **one formula per row** — a formula you can write in the leftmost period cell and drag right, unchanged, across all columns.
- Build **check rows** (balance sheet balances, sources = uses, sum-of-parts = total) that return `TRUE` / `OK` or a difference of zero, plus a master "error flag" cell at the top.
- Have **zero hardcodes inside formulas** — every driver sits in a clearly labelled input cell.
- Separate the model into **Inputs** → **Calculations** → **Outputs** so no sheet mixes assumptions with logic.
- Document assumptions, sources, and version so a reviewer needs no verbal walkthrough.

The bar is behavioural: could someone who has never spoken to you audit this in 20 minutes? If yes, you're proficient.

Hands-on: how to actually do it

1. Formatting conventions (the universal colour code). Apply font colours by cell type. This is muscle memory on every real desk:

Cell type	Font colour	Example
Hardcoded input / assumption	Blue (`RGB 0,0,255`)	Revenue growth `12%`
Formula / calculation	Black	`=D10*(1+D11)`
Link from another sheet (same file)	Green	`='P&L'!D25`
Link to another workbook	Red	dangerous — flag it
Hardcode that *must* stay in a formula	Highlight cell yellow	one-off overrides

Set blue fast: select input cells → `Ctrl+1` → Font → colour blue. Or record it to a shortcut. Reviewers scan for blue first — those are the only cells they need to challenge.

2. One formula per row. Never let column D's formula differ from column E's. Use absolute/relative references so a single dragged formula works across all periods:

```
' Row 12 = Revenue, Row 13 = growth %, dragged D⟶H unchanged:
=C12*(1+D13)
' C12 is prior period (relative), D13 is this period's growth input.
' NEVER: =C12*1.12 in D, =D12*1.10 in E  ← different formulas, unauditable
```

Anchor shared drivers with `$` : a WACC in `$B$4` referenced as `=D20/(1+$B$4)^D19` .

3. Kill hardcodes. Replace `=Sales*0.18` (GST buried in a formula) with an input cell:

```
' Bad:  =E30*0.18
' Good: E31 label "GST rate", B5 = 18%, formula =E30*$B$5
```

Audit an existing model for hidden constants — this flags any formula containing a raw number:

```
=IF(SUMPRODUCT(--ISNUMBER(--MID(FORMULATEXT(E30),ROW($1:$99),1)))>0,"CHECK","")
```

Simpler in practice: `Ctrl+~` toggles "show formulas" so you can eyeball the grid for stray digits, or use **Formulas → Trace Precedents** to see if a cell points nowhere.

4. Check rows. Add a dedicated check block. Core three-statement check — balance sheet must balance:

```
' Row 60: =Total Assets - (Total Liabilities + Total Equity)
=B45-(B52+B58)          ' should be 0 every period
```

Then a robust boolean using a tolerance (floating point can leave ₹0.0001):

```
=IF(ABS(B60)<0.5,"OK","ERROR")
```

Master flag at the top of every sheet (cell A1 area), aggregating all checks:

```
=IF(COUNTIF(Checks!$B$2:$B$40,"ERROR")>0,"⚠ MODEL BROKEN","✓ All checks pass")
```

Make errors impossible to miss with **Conditional Formatting**: select check row → Home → Conditional Formatting → New Rule → "Format cells that contain" → cell value not equal to 0 → fill red.

5. Stop broken inputs at the door with Data Validation. Select input cells → Data → Data Validation → Decimal, between 0 and 1 (for a %), with an input message. Growth of 1200% because you typed 12 instead of 0.12 gets rejected on entry.

6. Documentation. Every model gets a **Cover / Assumptions** sheet:

Field	Entry
Model name	FY26 Operating Budget
Author / owner	A. Sharma
Last updated	03-Jul-2026
Version	v2.3
Key assumptions	Rev growth 12%, GST 18%, WACC 13%
Sources	FY25 audited financials; RBI repo 6.5%
Colour key	Blue=input, Black=formula, Green=link

Add cell comments (`Shift+F2`) on any non-obvious driver, citing the source.

7. Version control. Excel isn't Git, but discipline substitutes. Save as `Budget_FY26_v2.3_2026-07-03.xlsx` — version + date in the filename, never "final_FINAL_v2". Keep a **Changelog** tab: date | version | who | what changed. For teams, store on SharePoint/OneDrive/Google Drive so version history is automatic and only one person edits at a time. For serious model shops, tools like Macabacus or Operis reconcile two file versions cell-by-cell.

Worked example / mini-project

Build a 3-year revenue-to-PAT projection for a Bengaluru SaaS firm and error-proof it.

Inputs sheet (all blue):

Driver	B (input)
FY25 Revenue (₹ Cr)	40
Revenue growth	25%
Gross margin	70%
Opex growth	18%
FY25 Opex (₹ Cr)	22
Tax rate	25%

Calc sheet — one formula per row, dragged across FY26–FY28:

```
Revenue (row 2):   =Inputs!$B$1*(1+Inputs!$B$2)     ' FY26; FY27 = C2*(1+Inputs!$B$2)
Gross profit (3):  =C2*Inputs!$B$3
Opex (4):          =Inputs!$B$5*(1+Inputs!$B$4)     ' FY26; then prior*(1+growth)
EBIT (5):          =C3-C4
Tax (6):           =C5*Inputs!$B$6
PAT (7):           =C5-C6
```

FY26 numbers: Revenue = 40 × 1.25 = ₹50 Cr; Gross profit = ₹35 Cr; Opex = 22 × 1.18 = ₹25.96 Cr; EBIT = ₹9.04 Cr; Tax = ₹2.26 Cr; **PAT = ₹6.78 Cr**.

Check sheet:

```
Margin sanity:   =IF(AND(Calc!C7<Calc!C5, Calc!C3>0),"OK","ERROR")  ' PAT < EBIT, GP positive
Recompute check: =IF(ABS(Calc!C5-(Calc!C3-Calc!C4))<0.01,"OK","ERROR")
Master flag:     =IF(COUNTIF(B2:B5,"ERROR")>0,"⚠ BROKEN","✓ PASS")
```

Now fat-finger growth to `250%` in the input cell — Revenue explodes to ₹140 Cr, but your Data Validation (0–1) blocks it, or the margin-sanity check stays OK while an eyeball tells you it's wrong. Change tax rate to `1.25` (typo) and the recompute check still passes but PAT goes negative — the "PAT < EBIT" check catches the sign flip. That's error-proofing earning its keep.

How it's tested

Practical assessment (the real filter): Firms give a timed 45–90 minute Excel test — often "build a 3-statement model from these raw financials" or "here's a broken model, find and fix the errors." They are watching for: colour-coding, no hardcodes, a balancing check row, and one-formula-per-row consistency. Many PE/IB shops hand you a deliberately broken model and time how fast you find the plug (a hardcoded cell breaking the balance).

Interview questions you should be able to answer cold:

- "How do you make sure a balance sheet balances in your model?" → check row = Assets − (Liab + Equity), tolerance-based OK/ERROR, master flag.
- "What's the difference between a blue and a black cell?" → input vs formula.

- "How do you avoid hardcoding?" → every driver in a labelled input cell; show formulas with `Ctrl+~` to audit.
- "You inherit a model — how do you audit it in 20 minutes?" → cover sheet, colour key, Trace Precedents/Dependents, check rows, Formula Auditing.
- "How do you version-control an Excel model?" → filename convention, changelog tab, SharePoint history, single editor.

Common mistakes & how pros avoid them

Mistake	Why it bites	Pro habit
Hardcode inside a formula (`=E10*1.08`)	Nobody can find/change the driver	All drivers in blue input cells
Different formula per column	One period breaks silently	One formula, dragged across
No check rows	Model balances by luck, then doesn't	Balance + sanity checks + master flag
`=A1=B1` exact-equality check	Floating point leaves ₹0.0001	`ABS(diff)<tolerance`
Mixing inputs, calcs, outputs on one sheet	Impossible to audit	Inputs → Calcs → Outputs separation
External workbook links	Break on the recipient's machine	Paste-special values or keep it self-contained; colour red
`final_v2_FINAL(3).xlsx`	Nobody knows the live version	`Name_vX.Y_YYYY-MM-DD` + changelog
No documentation	Every review needs a verbal walkthrough	Cover sheet + cell comments citing sources

Pros also press **F9 on a selected formula fragment** to evaluate just that piece, use **Evaluate Formula** (Formulas ribbon) to step through logic, and turn on **Error Checking** to catch inconsistent formulas (the little green triangles).

Learn-it roadmap & resources

Realistic time-to-proficiency: **3–4 weeks** of deliberate practice if you already know Excel formulas — this is discipline, not new functions.

- **Week 1:** Rebuild any past model applying the colour code + one-formula-per-row. Painful, permanent.
- **Week 2:** Add check rows and master flags to three-statement models. Learn Trace Precedents/Dependents, Evaluate Formula, `Ctrl+~`.
- **Week 3–4:** Do a timed "fix the broken model" drill; build a cover/changelog sheet as a template you reuse forever.

Resources: - **FAST Standard** (fast-standard.org) — free, the canonical model-building rulebook (Flexible, Appropriate, Structured, Transparent). - **CFI's** Financial Modeling & Valuation Analyst (FMVA) — paid, strong on formatting conventions and best-practice. - **Breaking Into Wall Street / Wall Street Prep** — paid,

industry-standard modeling courses used by Indian IB/PE hires. - **Macabacus** blog + Excel add-in (formatting shortcuts, model auditing) — free content, paid tool. - For CA/finance India roles, the **ICAI Excel and modeling** self-study material plus any three-statement template you rebuild by hand.

Quick-reference

Convention / tool	What to do
Blue font	Every hardcoded input / assumption
Black font	Formulas
Green font	Links from another sheet
Red font	External-workbook links (avoid)
One formula per row	Write once in first period cell, drag across
Anchor drivers	`$B$4` absolute for shared assumptions
Balance check	`=Assets-(Liab+Equity)` should be 0
Tolerance check	`=IF(ABS(diff)<0.5,"OK","ERROR")`
Master flag	`=IF(COUNTIF(checks,"ERROR")>0,"⚠ BROKEN","✓ PASS")`
Find hardcodes	`Ctrl+~` show formulas; Trace Precedents
Evaluate a fragment	Select in formula bar → `F9`
Guard inputs	Data → Data Validation (range limits)
Highlight errors	Conditional Formatting → red on ≠ 0
Cell comment	`Shift+F2` — cite the source
File naming	`Name_vX.Y_YYYY-MM-DD.xlsx` + changelog tab
Structure	Inputs → Calculations → Outputs (separate sheets)

Excel + AI (Copilot) and Automation

What it is & where it's used

This chapter is about making Excel do the boring 80% *for* you — so you spend your day on judgment, not grunt work. Three tools stack together:

1. **Copilot in Excel** — Microsoft's built-in AI. You type an instruction in plain English ("highlight rows where GST paid is above ₹50,000 and add a variance column") and it writes the formula, formats the range, or drafts an analysis.
2. **Macros / VBA** — recorded or hand-written scripts that replay clicks and keystrokes. This is how you turn a 40-minute month-end formatting ritual into one button press.
3. **LAMBDA** — Excel's native way to build your *own* reusable functions with no code, so a messy nested formula becomes `=NETSALARY(basic, hra)`.

Who pays for this: FP&A analysts (monthly MIS packs), audit/statutory teams (repetitive workpapers), tax associates (GST reconciliation, TDS working), accounts-payable teams (invoice cleanup), and anyone who produces the *same* report every month. In an India finance job, "the person who automated the MIS" gets noticed fast — because everyone else is still doing it by hand at 9 PM.

The gap: why companies want this (and college didn't teach it)

Your MBA taught you *what* a variance analysis means. It never taught you that in a real company you'll rebuild that variance report **12 times a year from a raw SAP/Tally dump that arrives in a slightly different shape each time**. The gap is *repetition at scale*.

College treats Excel as a calculator you use once for an assignment. Employers treat Excel as a **production line** — the same file, refreshed monthly, that six people depend on. Nobody taught you:

- That a recorded macro can eliminate the 30 clicks you do every single close.
- That AI can now write your `SUMIFS` faster than you can remember the argument order.
- That copy-pasting a giant nested formula into 200 cells is a *liability* — one edit and it silently breaks — whereas a named LAMBDA is auditable.

Companies want people who **remove work**, not people who heroically absorb it. That mindset shift is the real gap.

What "proficient" looks like

A job-ready person can, unaided:

- Use Copilot to generate, explain, and *sanity-check* a formula — and know when the AI is wrong.
- Record a macro for a repetitive formatting/cleanup task, then **open the VBA editor and edit it** (change a range, add a loop, wire it to a button).
- Judge **when to automate vs. when not to** (the "will I do this 3+ times?" test).
- Write a basic LAMBDA and save it as a Named Range so the whole team can use `=GSTSPLIT(...)`.
- Explain the risk: macros can't be undone with Ctrl+Z, so you always work on a copy.

The bar is *not* "expert VBA developer." It's "removes their own repetitive work and can read a macro someone else wrote."

Hands-on: how to actually do it

1. Copilot in Excel

Copilot lives on the **Home tab (rightmost)** in Microsoft 365. Your data must be a proper Table (`Ctrl+T`). Then click Copilot and type instructions:

```
Add a column called "GST Rate" that divides Tax Amount by Taxable Value, formatted as a percen
```

```
Show me total Taxable Value by State, sorted highest to lowest.
```

```
Highlight in red every row where Invoice Date is after the GST return due date.
```

Copilot returns the formula and offers to insert it. **Critical habit:** ask it to explain, then verify one row by hand. Example — you asked for a vendor-wise total and Copilot gives:

```
=SUMIFS(Invoices[Amount], Invoices[Vendor], [@Vendor], Invoices[FY], "2025-26")
```

Good prompt patterns that work reliably: - "Write an Excel formula to..." (it returns copy-usable syntax) - "Explain what this formula does: `=XLOOKUP( ... )` " - "What's wrong with this formula? It returns #N/A"

If you don't have Copilot licensed, the **free fallback** is ChatGPT/Gemini/Claude in a browser: paste your column headers and describe the goal — the formula logic is identical.

2. Record and edit a macro

Turn on the Developer tab: **File → Options → Customize Ribbon → tick Developer.**

Record: **Developer → Record Macro → name it** `FormatMIS` **→ do your steps → Stop Recording.**

Say you recorded formatting a report. Open **Developer → Visual Basic (Alt+F11)** to see and edit it:

```
Sub FormatMIS()
    ' Recorded: format the month-end MIS sheet
    Columns("A:F").AutoFit
    Range("A1:F1").Font.Bold = True
    Range("A1:F1").Interior.Color = RGB(0, 51, 102)
    Range("A1:F1").Font.Color = RGB(255, 255, 255)
    Columns("C:E").NumberFormat = "#,##0"   ' Indian thousands
    ActiveWindow.FreezePanes = True
End Sub
```

Now *edit* it — add a loop the recorder could never write, e.g. delete blank rows in a Tally export:

```
Sub CleanTallyDump()
    Dim i As Long, lastRow As Long
    lastRow = Cells(Rows.Count, "A").End(xlUp).Row
    ' Loop bottom-up so deleting a row doesn't skip the next one
    For i = lastRow To 2 Step -1
        If Trim(Cells(i, "A").Value) = "" Then
            Rows(i).Delete
        End If
    Next i
    MsgBox "Cleanup done. Rows now: " & Cells(Rows.Count, "A").End(xlUp).Row
End Sub
```

Wire it to a button: **Developer → Insert → Button (Form Control) → assign `CleanTallyDump`.** Save the file as **.xlsm** (macro-enabled) or macros vanish.

3. When to automate (the decision rule)

Situation	Automate?
One-off, never repeats	No — just do it
Same task 3+ times, stable steps	Yes — record a macro
Complex logic reused across files	Yes — LAMBDA or VBA
Steps change every time / need judgment	No — keep it manual
Touches money and can't be checked	Automate, but add a reconciliation check

Rule of thumb: **if (time saved per run × times per year) > (time to build × 2), automate it.**

4. LAMBDA basics

LAMBDA lets you name a formula. Instead of retyping a net-salary calc everywhere:

```
=LAMBDA(basic, hra, da, basic + hra + da − (basic*0.12))(50000, 20000, 5000)
```

Make it reusable: **Formulas → Name Manager → New → Name:** `NETSALARY`, Refers to:

```
=LAMBDA(basic, hra, da, basic + hra + da − (basic*0.12))
```

Now anyone types `=NETSALARY(50000, 20000, 5000)` → returns **68,000** (after 12% PF on basic). Another practical one — split a GST-inclusive amount into base + tax:

```
// Name: GSTBASE  — extract taxable value from an 18% inclusive amount
=LAMBDA(inclusive, rate, inclusive / (1 + rate))
// Usage: =GSTBASE(11800, 0.18)  →  10,000
```

Pair with `MAP` to apply across a range without dragging:

```
=MAP(A2:A100, LAMBDA(x, x/(1+0.18)))
```

Worked example / mini-project

Goal: Automate a monthly GST Input Tax Credit (ITC) reconciliation — purchase register vs. GSTR-2B.

Data (Purchase Register, sheet `PR`):

Vendor GSTIN	Invoice No	Taxable (₹)	IGST (₹)
29ABCDE1234F1Z5	INV-101	100000	18000
27PQRSX5678L1Z2	INV-102	50000	9000
29ABCDE1234F1Z5	INV-103	75000	13500

GSTR-2B (sheet `2B`) has the same columns as downloaded from the portal.

Step 1 — match with a formula. In `PR`, add a "Matched in 2B" column:

```
=IF(SUMIFS('2B'[IGST], '2B'[Invoice No], [@[Invoice No]], '2B'[Vendor GSTIN], [@[Vendor GSTIN]
```

Step 2 — a reusable LAMBDA for eligible ITC (block credit if unmatched):

```
// Name: ELIGIBLEITC
=LAMBDA(igst, status, IF(status="Matched", igst, 0))
// Usage in a column: =ELIGIBLEITC([@IGST], [@[Matched in 2B]])
```

Step 3 — a macro to produce the mismatch report every month with one click:

```
Sub GSTReconReport()
    Dim ws As Worksheet
    Set ws = Sheets("PR")
    ws.AutoFilterMode = False
    ' Filter to only MISMATCH rows
    ws.Range("A1").AutoFilter Field:=5, Criteria1:="MISMATCH"
    ws.Range("A1").CurrentRegion.Copy
    Sheets.Add.Name = "Mismatch_" & Format(Date, "MMM-YY")
    ActiveSheet.Paste
    Application.CutCopyMode = False
    ws.AutoFilterMode = False
    MsgBox "Mismatch report ready. Follow up with vendors before filing GSTR-3B."
End Sub
```

Result: what used to be an afternoon of VLOOKUPs and eyeballing is now — refresh the two sheets, click the button, chase the mismatches. On this data, INV-103 unmatched → ₹13,500 ITC blocked until the vendor uploads it. That number is exactly what a reviewer wants surfaced *before* filing.

How it's tested

Interview questions: - "Walk me through the last thing you automated in Excel. How much time did it save?" - "A macro deleted the wrong rows and there's no undo. What do you do / how do you prevent it?" (Answer: always run on a copy; version the file.) - "When would you *not* automate something?" - "Difference between a recorded macro and VBA you write yourself?" - "What is a LAMBDA and why use it over a nested formula?" (Answer: reuse, single point of edit, readability, auditability.)

Practical tests companies give: - **Timed cleanup screen:** "Here's a messy 5,000-row Tally/SAP export. Clean it and produce a vendor-wise summary in 20 minutes." They watch whether you do it by hand or reach for a macro/pivot. - **"Automate this" case:** given a repetitive monthly report, record a macro live on a shared screen. - **Formula-from-English:** "Write the formula for X" — increasingly they *allow* Copilot/AI and instead test whether you can **verify** the output.

Common mistakes & how pros avoid them

Mistake	How pros avoid it
Trusting Copilot/AI output blindly	Verify one row by hand; ask AI to explain its own formula
Running a macro on the only copy of live data	Always work on a duplicate file; macros ignore Ctrl+Z
Saving as .xlsx (macros silently lost)	Save as **.xlsm**
Hard-coding ranges (`A2:A500`) that break next month	Use Tables + structured references, or `End(xlUp)` to find last row
Automating a one-off	Apply the "3+ times" rule first
Giant nested formula copied 200 times	Convert to a named LAMBDA — one place to fix
Emailing .xlsm files (blocked by many gateways)	Zip it, or share via Teams/SharePoint
Deleting rows top-down in a loop (skips rows)	Loop **bottom-up** (`For i = lastRow To 2 Step -1`)

Learn-it roadmap & resources

Time to job-ready: 3–4 weeks, ~1 hour/day.

- **Week 1 — Copilot / AI formulas.** Practice prompting for `SUMIFS` , `XLOOKUP` , conditional formatting. If unlicensed, use free ChatGPT/Gemini. Goal: never get stuck on syntax again.
- **Week 2 — Record & run macros.** Automate your own recurring formatting task. Learn to save .xlsm and assign buttons.
- **Week 3 — Edit VBA.** Learn variables, `For` loops, `If` , `MsgBox` , `Cells` / `Range` , and `End(xlUp)` . Build the GST-recon macro above.
- **Week 4 — LAMBDA + MAP.** Convert your three most-used nested formulas into named functions.

Resources (mostly free): - Microsoft Learn — "Automate tasks with the Macro Recorder" and "LAMBDA function" docs. - ExcelIsFun and Leila Gharani (YouTube) — best free VBA and LAMBDA channels. - Chandoo.org — India-friendly Excel automation examples. - **Certification:** Microsoft Office Specialist (MOS): Excel Expert (MO-201) validates macros/LAMBDA on a CV; the free skills matter more than the badge in India.

Quick-reference

Task	How
Open Copilot	Home tab → Copilot (M365)
Turn on macros	File → Options → Customize Ribbon → Developer
Record macro	Developer → Record Macro
Open code editor	`Alt+F11`
Save with macros	Save as **.xlsm**
Find last row (VBA)	`Cells(Rows.Count,"A").End(xlUp).Row`
Loop that deletes rows	`For i = lastRow To 2 Step -1`
Message box	`MsgBox "text"`
Name a LAMBDA	Formulas → Name Manager → New
LAMBDA syntax	`=LAMBDA(a, b, a+b)(1, 2)`
Apply over a range	`=MAP(range, LAMBDA(x, x*1.18))`
GST base from inclusive	`=inclusive/(1+rate)`
Automate decision	(time saved/run × runs/yr) > (build time × 2)

Golden rules: verify AI output, work on a copy (no undo for macros), save as .xlsm, and only automate what you'll repeat 3+ times.

The interview Excel & modeling test

What it is & where it's used

The interview Excel/modeling test is a **timed, hands-on assessment** where you build or fix something live — no ChatGPT, no calling a friend, often no internet. It is the single most common filter between "sounds good on the CV" and an offer for roles in **investment banking, private equity, equity research, corporate FP&A, transaction advisory / valuations, credit analysis, and startup finance (fundraise/CFO-track)**.

Formats you will meet in India and globally:

Format	Duration	Where
Live shared-screen build (Zoom + Excel)	30–90 min	Most banks, PE, boutique advisory
Take-home model (emailed, deadline)	3–24 hrs	Startups, mid-market PE, research
"Fix my broken model" / audit	20–45 min	FP&A, controllership
Aptitude + Excel combo (SHL/Kenexa style)	45–60 min	BPO/KPO, Big 4 analytics, GCC roles

For an India-based candidate targeting Big 4 (Deloitte/PwC/EY/KPMG), GCCs (Wells Fargo, JPMC, Goldman Bengaluru/Hyderabad), or a domestic AMC/broker, expect the "fix + build" variant far more than a full LBO.

The gap: why companies want this (and college didn't teach it)

Your MBA taught you what NPV *means*. It did not make you build a 3-statement model that **balances**, under time pressure, with a keyboard-only workflow, correct sign conventions, and a circularity toggle for interest-on-average-debt. CA Inter drilled accounting rules but on paper, not linked in Excel where the P&L flows to retained earnings which flows to the balance sheet.

The gap employers pay to close:

- **Speed under the keyboard** — pros navigate with `Alt` shortcuts, not the mouse. A mouse-user is spotted in 30 seconds.
- **Structure** — inputs (blue), formulas (black), links (green), clearly separated. College spreadsheets are one giant messy tab.
- **A model that ties** — Assets = Liabilities + Equity to the rupee, cash flow reconciles to the balance sheet cash line.
- **Judgment** — sensible assumptions with a stated rationale, not a random 10% growth.

What "proficient" looks like

An interviewer's mental checklist for a "job-ready" candidate:

1. Builds a clean **3-statement model** from a blank sheet in ~45 min, statements linked, BS balancing.
2. **Keyboard-first**: no mouse for navigation, formatting, or fill.
3. Uses **relative/absolute references** correctly (`$` discipline) and **named ranges** or structured refs where sensible.
4. Handles **circularity** (interest ↔ cash ↔ debt) knowingly — iterative calc on, or a circ-breaker switch.

5. Writes **auditable** formulas: no hardcodes inside formulas, no `=B2+B3+B4` where `SUM` belongs, colour-coded inputs.
6. Can **stress a driver** (revenue −20%) and immediately explain the covenant/cash impact.
7. Talks while building — narrates assumptions.

Hands-on: how to actually do it

Keyboard shortcuts that signal "pro" (Windows)

```
Ctrl + ~            Toggle show-all-formulas (audit view)
Alt + =             AutoSum
F2                  Edit cell / F4 = toggle $ absolute
Ctrl + Shift + →/↓  Select to edge of data
Alt + E, S, V       Paste Special > Values (older) / Ctrl+Alt+V
Alt + H, O, I       Autofit column width
Ctrl + [            Jump to precedent cell (trace)
F9                  Recalculate (and evaluate part of a formula)
Alt + A, T          Add filter
```

The lookup you MUST use

```
=XLOOKUP(lookup_value, lookup_array, return_array, "Not found", 0)
```

Legacy fallback interviewers still test:

```
=INDEX($D$2:$D$500, MATCH(A2, $B$2:$B$500, 0))
```

`VLOOKUP` is acceptable but flag that it breaks on column insertion — `INDEX/MATCH` or `XLOOKUP` shows maturity.

Core modeling formulas

```
Revenue growth:        =Prev_Rev * (1 + growth_%)
Depreciation (SLM):    =(Cost - Salvage) / Useful_life
Ending debt:           =Opening_debt + Drawdown - Repayment
Interest (on avg):     =Rate * AVERAGE(Opening_debt, Closing_debt)
Retained earnings:     =Opening_RE + PAT - Dividends
Cash (from CFS):       =Opening_cash + CFO + CFI + CFF
Balance check:         =Total_Assets - (Total_Liab + Total_Equity)   'must = 0
```

Circularity switch (interest on average debt creates a loop)

Put a toggle in one cell `Circ_Switch` (1/0):

```
=IF($Circ_Switch=1, Rate*AVERAGE(Open_debt,Close_debt), Rate*Open_debt)
```

And enable **File > Options > Formulas > Enable iterative calculation** (100 iterations, 0.001). Narrate this — it is a classic "do they even know?" test.

Error-proofing

```
=IFERROR(Sales/Units, 0)
=SUMIFS(Amount, Region, "West", Month, "≥"&DATE(2026,4,1))
```

Worked example / mini-project

Prompt (45 min): "Build a 3-year projection for a mid-market manufacturer. FY26 revenue ₹500 cr, growing 12% then 10%. EBITDA margin 18%. Term loan ₹200 cr at 10%, ₹40 cr repaid annually. Tax 25%. Show that the balance sheet balances."

Build a driver block, then link:

Line (₹ cr)	FY26	FY27	FY28	Formula (FY27)
Revenue	500	560	616	`=E_rev*(1+0.12)`
EBITDA (18%)	90.0	100.8	110.9	`=Rev*0.18`
Depreciation	20	20	20	input
EBIT	70.0	80.8	90.9	`=EBITDA-Dep`
Interest (10% avg)	18.0	16.0	12.0	`=0.10*AVERAGE(OpenDebt,CloseDebt)`
PBT	52.0	64.8	78.9	`=EBIT-Int`
Tax @25%	13.0	16.2	19.7	`=PBT*0.25`
PAT	39.0	48.6	59.2	`=PBT-Tax`
Closing debt	160	120	80	`=OpenDebt-40`

Then the tie-out that wins the round:

```
Cash_close = Cash_open + PAT + Dep − Debt_repay − Capex + Δ Working_capital
Balance_check = Total_Assets − Total_Liab_Equity   → 0.00
```

If `Balance_check` isn't zero, **stop and trace with `Ctrl+[`** before adding anything else. A model that doesn't balance but looks pretty scores lower than a simpler one that ties.

How it's tested

The practical test itself — a typical live shared-screen script: 1. "Here's a blank sheet. Build me a revenue build with these drivers." (structure + speed) 2. "Now link it to a simple P&L." (references, sign convention) 3.

"Add debt with interest on average balance — make it work." (circularity) 4. "The balance sheet is off by ₹4 cr, find it." (audit under pressure) 5. "Flex revenue down 20% — what breaks?" (judgment)

The take-home — you get an .xlsx of a real-ish company; deliver a model + a 5-line email summarising the investment view. They grade formatting and the *email* as much as the math.

Verbal questions fired mid-build: - "Walk me through a 3-statement model — if depreciation goes up ₹100, what happens to all three statements?" (Answer: P&L PAT −75 at 25% tax; CFS add back +100 so cash +25; BS: cash +25, PP&E −100, RE −75 → still balances.) - "How do you break circularity?" / "Difference between deferred tax asset and provision?" - "Why is my cash flow not matching the balance sheet?"

GCC/Big-4 aptitude combo: SHL-style timed Excel — pivot a 5,000-row table, `SUMIFS` a summary, spot the duplicate. Practise pivots and `SUMIFS` cold.

Common mistakes & how pros avoid them

Mistake	What pros do
Reaching for the mouse	Keyboard-only; learn 15 shortcuts to reflex
Hardcoding numbers inside formulas (`=B2*1.12`)	Growth rate lives in its own labelled input cell
No colour coding	Blue = input, black = calc, green = cross-sheet link
Building fancy tabs before it balances	Get BS to tie *first*, decorate later
Silent on assumptions	Narrate: "I'll assume WC stays 15% of sales because…"
Panicking when balance check ≠ 0	Trace precedents `Ctrl+[` ; check sign on one line at a time
Circular ref error ignored	Toggle iterative calc, explain the loop out loud
Over-engineering under time pressure	Deliver a working simple model over a broken complex one

Red flags interviewers actively watch for: mouse-dragging to select, retyping the same number in multiple cells, `=A1+A2+A3+A4` instead of `SUM` , a model that doesn't balance and the candidate not noticing, freezing silently when stuck, and formatting a chart before the numbers work.

Learn-it roadmap & resources

Realistic time to test-ready: 4–6 weeks of ~1 hr/day if you already know accounting (you do, from CA Inter).

Week	Focus
1	Keyboard shortcuts + `XLOOKUP` / `INDEX-MATCH` / `SUMIFS` , drilled daily
2	Single-statement builds; `$` discipline; colour convention
3	Full 3-statement link; get the BS to balance from scratch 5×
4	Circularity, debt schedule, sensitivity tables (`Data > What-If`)
5–6	Timed mocks: 45-min builds, then "find the error" drills

Resources: - **Breaking Into Wall Street (BIWS)** / **Wall Street Prep** / **CFI** — paid, the industry standard 3-statement + LBO courses. - **Macabacus** and **Corporate Finance Institute** free formula/shortcut guides. - Free: rebuild a listed company's model from its annual report (screener.in gives Indian financials free). - Certification: **CFI FMVA** (globally recognised, ~₹25–40k) or **NISM** for domestic markets roles; BIWS for banking.

Practice the *timed* condition — set a 45-minute timer and build from blank. Untimed practice does not prepare you for the thing being tested: performance under the clock.

Quick-reference

```
BUILD ORDER:  Assumptions → Revenue → P&L → Debt schedule → CFS → BS → Balance check
BALANCE:      Total Assets  =  Total Liabilities + Equity   (to the rupee)
CASH TIE:     Cash_close = Cash_open + CFO + CFI + CFF
DEP TEST:     +100 Dep → PAT −75, Cash +25, PP&E −100, RE −75  (BS still ties)
```

Need	Formula
Lookup	`=XLOOKUP(val, arr, ret, "NA", 0)`
Robust lookup	`=INDEX(ret,MATCH(val,key,0))`
Conditional sum	`=SUMIFS(amt, crit_rng, crit)`
Safe divide	`=IFERROR(a/b,0)`
Interest (avg)	`=rate*AVERAGE(open,close)`
Retained earnings	`=open_RE + PAT − div`
Balance check	`=Assets − (Liab + Equity)` → 0

Colour convention	Meaning
Blue font	Hardcoded input / assumption
Black font	Formula within the sheet
Green font	Link from another sheet
Red font	Warning / external link / check

Top shortcuts: `F2` edit · `F4` toggle `$` · `Alt+=` sum · `Ctrl+[` trace precedent · `Ctrl+~` show formulas · `Ctrl+Shift+arrow` select range · `Ctrl+Alt+V` paste special.

Cheat-sheet: formulas & shortcuts

What it is & where it's used

This chapter is the printable last-page you rip out and tape next to your monitor. It compresses the whole guide — lookups, aggregation, cleanup, dates, finance math, keyboard flow, PivotTables, a bit of SQL and Python — into a dense, copy-usable reference. Every analyst, accountant, FP&A associate, tax executive and equity researcher lives in these formulas eight hours a day. In an interview you get judged on how few keystrokes you waste; on the job you get paid for turning a raw dump into a clean answer before the 6 pm review call.

Roles that need it: **FP&A / budgeting**, **financial analyst / equity research**, **accounts & controllership**, **audit**, **GST/tax compliance**, **investment banking associate**, **credit analyst**. If Excel is open on your screen more than your email, this page is your operating system.

The gap: why companies want this (and college didn't teach it)

MBA and CA classrooms teach *concepts* — NPV, working capital, revenue recognition — but assume the numbers are already in a tidy table. Industry never hands you a tidy table. You get a 40,000-row SAP export with trailing spaces, dates stored as text, ₹ signs glued to amounts, and a "Customer Name" column that's spelled three different ways. The gap is **mechanical fluency**: the ability to reconcile, join, clean and summarise without touching the mouse and without breaking when a row is added.

College rewards the *right final number*. Employers reward *the right number, auditable, refreshable, and delivered in twenty minutes*. This cheat-sheet is the muscle memory that closes that gap.

What "proficient" looks like

A job-ready person can, unaided:

- Build a two-condition lookup with `XLOOKUP` / `INDEX–MATCH` without converting anything to a helper column.
- Reconcile two ledgers (book vs bank, GSTR-2B vs purchase register) and flag the breaks.
- Clean a dirty export — `TRIM`, `TEXT`-to-date, split names — in under five minutes.
- Drive Excel keyboard-only: `Ctrl+Shift+Arrow`, `Alt+=`, `Ctrl+T`, `F4`.
- Spin up a PivotTable and a `SUMIFS` dashboard that refresh when data grows.
- Know when to leave Excel — pull the raw data with a `SELECT … GROUP BY` instead of a 500k-row copy-paste.

Hands-on: how to actually do it

Lookups & joins

```
' Exact lookup, modern (Excel 365 / 2021)
=XLOOKUP(A2, Master[ID], Master[Name], "Not found")

' Two-condition lookup (invoice + line item), array-safe
=XLOOKUP(1, (Master[Inv]=A2)*(Master[Line]=B2), Master[Amount])

' Legacy but universal — works everywhere incl. old corporate builds
=INDEX(Master[Name], MATCH(A2, Master[ID], 0))

' Left-lookup / approximate band (tax slab, ageing bucket) — sorted ascending
=XLOOKUP(A2, Slab[Upper], Slab[Rate], , 1)   ' 1 = next larger match
```

Avoid `VLOOKUP` for new work: it breaks when columns are inserted and can't look left. If you must, lock the table: `=VLOOKUP(A2,$D$2:$F$999,3,FALSE)`.

Conditional aggregation (the FP&A workhorse)

```
=SUMIFS(Amt, Region, "West", Month, "≥"&DATE(2026,4,1))
=COUNTIFS(Status, "Open", Ageing, ">90")
=AVERAGEIFS(Margin, Product, "SKU-101")
=SUMPRODUCT((Region="West")*(Qty)*(Price))   ' when SUMIFS can't multiply
```

Cleanup & text

```
=TRIM(CLEAN(A2))                       ' strip spaces + non-printing chars
=VALUE(SUBSTITUTE(A2,"₹",""))          ' text "₹1,200" → number
=DATEVALUE(TEXT(A2,"dd-mm-yyyy"))      ' text date → real date
=TEXTSPLIT(A2," ")                     ' 365: split "Ravi Kumar" into cells
=TEXTBEFORE(A2,"@")  =TEXTAFTER(A2,"@")
=PROPER(TRIM(A2))                      ' normalise names
=IFERROR(XLOOKUP( ... ), "check")      ' never show #N/A in a deck
```

Dates & finance math

```
=EOMONTH(A2,0)                          ' month-end (period close)
=EDATE(A2,12)                           ' same day next year
=YEARFRAC(A2,B2,1)                      ' actual/actual for interest
=NPV(0.12, C2:C11) + C1                 ' C1 = year-0 outflow
=XNPV(0.12, Cash, Dates)                ' irregular cash flows
=IRR(C1:C11)   =XIRR(Cash, Dates)
=PMT(0.09/12, 60, -500000)              ' EMI on ₹5,00,000, 9%, 5 yrs
=FV(0.07,10,-100000)   =PV(0.08,5,,-1000000)
```

SQL — pull, don't copy-paste

```
-- Monthly sales by region, straight from the source
SELECT region, DATE_TRUNC('month', invoice_date) AS mth,
       SUM(taxable_value)  AS sales,
       SUM(igst+cgst+sgst) AS tax
FROM   sales_register
WHERE  invoice_date ≥ '2026-04-01'
GROUP  BY region, mth
ORDER  BY mth, sales DESC;
```

Python — when the file is too big for Excel

```
import pandas as pd
df = pd.read_excel("sap_dump.xlsx")
df["amount"] = (df["amount"].astype(str)
                .str.replace("₹","").str.replace(",","").astype(float))
pivot = df.pivot_table(index="region", columns="month",
                       values="amount", aggfunc="sum", margins=True)
pivot.to_excel("summary.xlsx")
```

DAX (Power BI / Power Pivot)

```
Total Sales   = SUM(Sales[Amount])
Sales LY      = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
YoY %         = DIVIDE([Total Sales]-[Sales LY], [Sales LY])
Margin %      = DIVIDE([Total Sales]-[Total COGS], [Total Sales])
```

Worked example / mini-project

GSTR-2B vs Purchase Register reconciliation — the monthly task every tax executive does.

You have two sheets: `Books` (your recorded purchases) and `GSTR2B` (what the portal says your vendors filed). Goal: find Input Tax Credit (ITC) you can safely claim.

Sample rows:

GSTIN	Invoice	Taxable (₹)	IGST (₹)
27ABCDE1234F1Z5	INV-091	1,00,000	18,000
29PQRSX9876G1Z2	INV-104	50,000	9,000

Build a match key and lookup in `Books`:

```
' Key in both sheets
=TRIM(A2)&"|"&TRIM(B2)

' In Books, pull the 2B tax against each invoice
=XLOOKUP([@Key], GSTR2B[Key], GSTR2B[IGST], "Not in 2B")

' Break flag
=IF([@BooksIGST]=[@IGST_2B], "Match",
   IF([@IGST_2B]="Not in 2B","Missing in 2B","Value diff"))

' Claimable ITC = only what appears in 2B
=SUMIFS(Books[IGST], Books[Status], "Match")
```

Then a PivotTable on `Status` gives the summary the manager wants:

Status	Count	IGST (₹)
Match	142	11,84,000
Missing in 2B	8	61,200
Value diff	3	4,500

Claimable this month = ₹11,84,000. The ₹61,200 "Missing in 2B" is chased with vendors — that's the deliverable. Whole thing: 15 minutes once the keys are built.

How it's tested

Timed Excel test (most common, 30–45 min): given a raw sheet, you're asked to (1) clean it, (2) build a lookup between two tables, (3) produce a `SUMIFS` summary or PivotTable, (4) sometimes an EMI/NPV. Graders watch whether you use structured references and keyboard shortcuts — mouse-heavy candidates run out of time.

SQL screen (analyst/BI roles): "Write monthly revenue by region" or "top 5 customers by outstanding" — a `GROUP BY` + `ORDER BY ... LIMIT`, sometimes a `JOIN`.

Case / "close these books": a mini reconciliation (bank vs book, or 2B) with intentional breaks planted. They want to see you *find* the breaks, not just tie the total.

Interview questions you'll hear: - "VLOOKUP vs INDEX-MATCH vs XLOOKUP — when and why?" - "Your SUMIFS returns 0 but you know data exists — debug it." (Answer: text-vs-number mismatch, trailing spaces, wrong criteria range.) - "Difference between NPV and XNPV / IRR and XIRR?" - "How do you make a report auto-refresh when rows are added?" (Answer: Excel Tables + structured refs, or Power Query.)

Common mistakes & how pros avoid them

Mistake	Fix
`SUMIFS` returns 0 — numbers stored as text	`=A2+0` or `VALUE()`, or Data > Text to Columns
`VLOOKUP` breaks after column insert	Use `XLOOKUP` / `INDEX-MATCH`
Forgetting `$` locks, formula drifts	`F4` to toggle absolute refs
`NPV()` including year-0 in the range	Keep year-0 outside: `=NPV(r,C2:C11)+C1`
Merged cells killing sort/pivot	Never merge in data ranges; use Center Across Selection
Hard-coding numbers inside formulas	Put assumptions in labelled input cells
Ranges not growing with data	Convert to Table (`Ctrl+T`) so refs auto-extend
Comparing dates as text	Parse with `DATEVALUE` / `TEXT` first
Circular reference in interest calc	Enable iterative calc knowingly, or restructure

Pros also **audit before they trust**: `Ctrl+[` to trace precedents, `F9` to evaluate a selected formula chunk, and a `SUM` cross-check at the bottom of every reconciliation.

Learn-it roadmap & resources

Realistic time to interview-ready fluency: **6–8 weeks** at 1 hr/day if you already know finance concepts.

- **Weeks 1–2:** Lookups, SUMIFS/COUNTIFS, Tables, keyboard shortcuts. Rebuild a real bank statement into a summary.
- **Weeks 3–4:** Text cleanup, dates, IFERROR, PivotTables. Do the GSTR-2B reco above with your own data.
- **Weeks 5–6:** Finance functions (NPV/IRR/PMT), then Power Query + one SQL `GROUP BY`.
- **Weeks 7–8:** Basic Python/pandas for big files, and DAX if the role touches Power BI.

Resources: **ExcelJet** (free formula reference), **Chandoo.org** (dashboards), **Microsoft Learn** (Power Query + DAX, free), **Mode SQL Tutorial** (free), **Corporate Finance Institute (CFI)** — paid but recognised for FMVA. Certifications worth the line on a resume: **Microsoft Office Specialist: Excel Expert**, **CFI FMVA**, **Google/DataCamp SQL** for BI-leaning roles.

Quick-reference

Top keyboard shortcuts

Shortcut	Action
`Ctrl+Shift+↓/→`	Select to end of data
`Alt+=`	AutoSum
`Ctrl+T`	Convert range to Table
`F4`	Toggle `$` / repeat last action
`Ctrl+;`	Insert today's date
`Ctrl+Shift+L`	Toggle filters
`Ctrl+D` / `Ctrl+R`	Fill down / right
`Alt+H+O+I`	Autofit column width
`F2`	Edit cell / `F9` evaluate selection
`Ctrl+[`	Trace precedents
`Ctrl+PgUp/PgDn`	Move between sheets
`Ctrl+Shift+V`	Paste Special (values)

Top formulas

Need	Formula
Lookup	`=XLOOKUP(k, ids, vals, "NA")`
2-key lookup	`=XLOOKUP(1,(a=x)*(b=y),vals)`
Conditional sum	`=SUMIFS(amt, reg,"West", mth,"≥"&d)`
Count	`=COUNTIFS(rng,">90")`
Clean text	`=TRIM(CLEAN(A2))`
₹text→number	`=VALUE(SUBSTITUTE(A2,"₹",""))`
Month-end	`=EOMONTH(A2,0)`
EMI	`=PMT(r/12,n,-P)`
Irregular NPV/IRR	`=XNPV(r,cf,dt)` / `=XIRR(cf,dt)`
No errors in deck	`=IFERROR(x,"check")`

SQL skeleton: `SELECT dim, SUM(x) FROM t WHERE d≥'2026-04-01' GROUP BY dim ORDER BY 2 DESC LIMIT 5;`

pandas skeleton: `df.pivot_table(index="region", values="amt", aggfunc="sum", margins=True)`